Administración de Proyectos de Software

Resumen final para el examen teórico: todo el programa en un solo documento

Administración de Proyectos de Software (7502) — UNS

Examen: 27 de octubre de 2026

De dónde sale este documento. Es la síntesis de los siete resúmenes teóricos de los TP0 a TP6 (`tpN/material-de-estudio/resumen/`), filtrada contra las **diapositivas de la cátedra** (Estévez – Fillottrani, 2026), los enunciados de los TPs y las resoluciones publicadas por la cátedra. Es **autocontenido**: está todo lo que puede tomarse en el examen (todas las preguntas de los TPs, los procedimientos de los ejercicios y los ejemplos resueltos por la cátedra). Se dejaron afuera las ampliaciones de los libros. La única excepción son los temas que el TP6 pregunta y que no tienen diapositivas (las métricas de Fenton y la auditoría de sistemas): esos salen de la bibliografía que cita el TP6, y están marcados. Cada sección indica entre corchetes el resumen original, por ejemplo **[TP3]**, por si alguna vez querés más detalle.

Qué acompaña a este resumen. La *guía de ejercicios* (`examen/practica/ejercicios-examen.pdf`), con plantillas para cada tipo de ejercicio; sus *respuestas* (`respuestas-ejercicios.pdf`); y el *quiz interactivo* (`examen/quiz/index.html`), con preguntas de opción múltiple por unidad.

Qué tiene más chances de aparecer

- **Prácticos** (según el docente, "muy similares a los de los TPs"): red PDM con planificación temprana y tardía, ventanas y márgenes (§7.6); Puntos de Función (§8.2); matriz persona/perfil y recursos críticos (§9.4); selección de historias por velocity (§2); comparación de soluciones (§5.4); y, a partir de un enunciado largo, sacar **requerimientos** (§5) y **riesgos** con tabla, línea de corte y RSGR (§6). *No* se toma armar un POS ni un COS completo, pero sí hay que saber qué son.
- **Teóricos**: las "preguntas de conceptos importantes" de cada TP. Están todas cubiertas acá.
- **Fórmulas y plantillas**: según el docente, *toda fórmula o plantilla necesaria viene en el enunciado*. No hace falta memorizar pesos ni coeficientes: lo que se evalúa es saber *qué significa* cada término, *cuándo* se aplica y *cómo interpretar* el resultado. La hoja de fórmulas (§12) está para que te familiarices con la notación.

Índice

1 El mapa de la materia **3**

2 Metodologías ágiles y Scrum **3**
2.1 De dónde vienen las metodologías ágiles 3
2.2 Scrum: conceptos, roles, artefactos y reuniones 4
2.3 Historias de usuario, estimación y velocity 5

3 Proyectos: definición, parámetros, clasificación y etapas **6**
3.1 Qué es un proyecto y qué es administrarlo 6
3.2 Parámetros, triángulo de alcance y corrimientos 7
3.3 Éxito y fracaso 7
3.4 Clasificación de proyectos (A–D) 7
3.5 Etapas, hitos y ciclos de vida 8
3.6 Atributos críticos 8

4 Concepción: COS, POS y aprobación . . . **9**
4.1 La falla de comunicación cliente–proveedor . . . 9
4.2 Condiciones de Satisfacción (COS) . . . 9
4.3 El POS (Project Overview Statement) . . . 9
4.4 Aprobación para la planificación . . . 10

5 Requerimientos y soluciones . . . **11**
5.1 Requerimientos: definición, proceso y tipos . . . 11
5.2 Funcionales: separar función, calidad y solución . . . 11
5.3 No funcionales: atributos, escala, test y niveles . . . 12
5.4 Soluciones . . . 13

6 Gestión de riesgos . . . **14**
6.1 Qué es un riesgo y cómo se clasifica . . . 14
6.2 Herramientas para identificarlos . . . 15
6.3 Evaluación: tabla de riesgos, línea de corte y matrices . . . 16
6.4 El plan RSGR y su costo . . . 17

7 Planificación . . . **17**
7.1 Para qué planificar y la JPPS . . . 17
7.2 WBS (Estructura de Descomposición del Trabajo) . . . 18
7.3 Estimación: duración, esfuerzo, recursos y costos . . . 18
7.4 Diagramas: Gantt, AON (PDM) y AOA (PERT) . . . 20
7.5 Dependencias, restricciones y lags . . . 20
7.6 Planificación temprana, tardía, ventana, márgenes y camino crítico . . . 21

8 Métricas: Puntos de Función y COCOMO . . . **23**
8.1 Por qué medir la especificación . . . 23
8.2 Puntos de Función de Albrecht . . . 23
8.3 COCOMO 81 . . . 24
8.4 COCOMO II y otros modelos . . . 26

9 Lanzamiento: equipo, reglas, cambios y kick-off . . . **26**
9.1 Organización del equipo . . . 27
9.2 Reglas de trabajo en grupo . . . 28
9.3 Administración de cambios de alcance . . . 28
9.4 Paquetes de trabajo, cronograma final y matriz por perfil . . . 29
9.5 La reunión de kick-off . . . 29

10 Monitoreo, control y cierre . . . **30**
10.1 Monitoreo y control . . . 30
10.2 Cierre del proyecto . . . 31

11 Calidad: Fenton, ISO, CMM y auditoría . . . **31**
11.1 Atributos externos y definiciones de calidad . . . 31
11.2 Defectos, densidad, spoilage, usabilidad, mantenimiento y productividad . . . 32
11.3 Normas ISO . . . 33
11.4 CMM y CMMI . . . 34
11.5 Auditoría de sistemas de información . . . 35

12 Hoja de fórmulas . . . **37**

13 Siglas y preguntas de repaso . . . **37**

1 El mapa de la materia

Toda la materia se organiza alrededor del **ciclo de vida de un proyecto**. Cada unidad (y cada TP) profundiza una etapa, y las metodologías ágiles son la forma alternativa de recorrerlo en ciclos cortos.

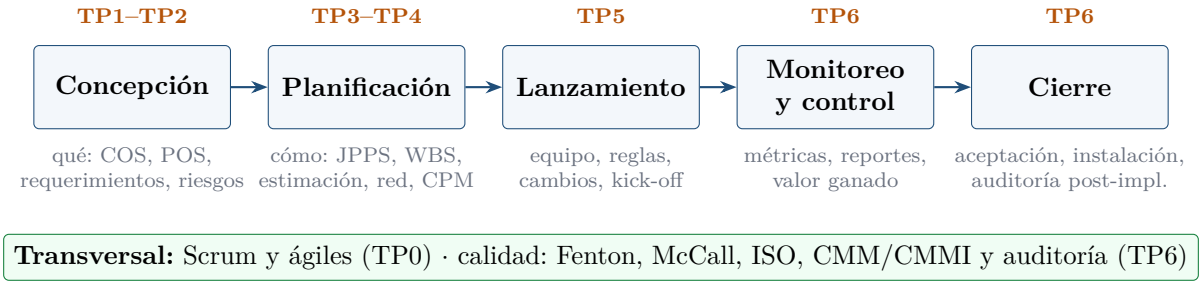


Figura 1: Las cinco etapas de un proyecto y qué TP trabaja cada una. No todo proyecto las recorre todas (§3.5).

2 Metodologías ágiles y Scrum [TP0]

2.1 De dónde vienen las metodologías ágiles

A fines de los 60 se habló de la **crisis del software** (1968): la demanda, la complejidad y los desafíos de los sistemas crecían mucho más rápido que la capacidad de construirlos, porque se seguía trabajando con la *misma gente*, los *mismos métodos* y las *mismas herramientas*. La respuesta fue *gestionar*, y se gestionó con la lógica de la ingeniería civil: el modelo en **cascada** (requerimientos → diseño → implementación → verificación → mantenimiento), donde cada etapa termina y entrega un documento a la siguiente.

Funcionó en los 70, cuando los negocios cambiaban lento. En los 90 la globalización y la aceleración de la información acortaron los ciclos y volvieron cambiantes a los clientes. En cascada eso es letal: el cliente **ve el producto recién al final**, y el costo de corregir un requisito mal entendido crece cuanto más tarde se descubre. Además, el proceso se había llenado de papeles. Distintos grupos ensayaron formas más livianas (Scrum, XP, FDD, Kanban) y en **febrero de 2001**, en Snowbird (Utah), diecisiete referentes firmaron el **Manifiesto Ágil**.

El agilismo no es una metodología

Es un conjunto de **valores y principios**. Scrum, XP, Kanban o Lean son implementaciones concretas de esos valores (se dibuja como un paraguas "Agile" con las metodologías debajo).

Manifiesto Ágil: los cuatro valores

Individuos e interacciones sobre procesos y herramientas; **software funcionando** sobre documentación extensiva; **colaboración con el cliente** sobre negociación contractual; **respuesta ante el cambio** sobre seguir un plan. *Aunque valoramos los elementos de la derecha, valoramos más los de la izquierda.*

La última frase es la que más se olvida: no se descarta la documentación ni el plan; se establece qué pesa más cuando hay que elegir. Los **doce principios** bajan los valores a la práctica:

- *Entrega de valor*: satisfacer al cliente con entregas tempranas y continuas; entregar software funcional cada 2 semanas a 2 meses (mejor más corto); el software funcionando es la medida principal de progreso.
- *Cambio*: aceptar requisitos cambiantes, incluso tarde, como ventaja competitiva para el cliente.
- *Personas*: negocio y desarrolladores trabajan juntos a diario; proyectos en torno a individuos

motivados a los que se les confía el trabajo; conversación cara a cara; ritmo sostenible (sin horas extra crónicas).
- *Calidad*: atención continua a la excelencia técnica; **simplicidad** ("maximizar el trabajo no hecho": no construir lo que nadie pidió); las mejores arquitecturas surgen de equipos autoorganizados.
- *Mejora*: a intervalos regulares, el equipo reflexiona y ajusta (es la retrospectiva).

¿Ágil o tradicional? (Boehm y Turner). Se marca el proyecto sobre varios ejes; si queda cerca del centro, es terreno ágil. Ágil conviene con equipos **chicos** (3 a 10), mucho personal **senior**, requisitos que cambian mucho (30–50 % por mes), producto de **baja criticidad** (un defecto cuesta comodidad o dinero discrecional, no vidas), cultura que prospera en el caos, equipo **poco disperso** y **estable**. El caso opuesto (cientos de personas, muchos juniors, requisitos estables, vidas en juego, equipos repartidos en varios países) pide un enfoque tradicional, guiado por un plan.

2.2 Scrum: conceptos, roles, artefactos y reuniones

Scrum (Jeff Sutherland lo aplica en 1993, Ken Schwaber lo formaliza en 1995; el nombre viene de la formación del rugby) es un **framework de gestión**, no una metodología completa: define roles, reuniones y artefactos, pero no dice cómo programar ni probar. Busca maximizar el retorno de la inversión, trabajar en ciclos con software entregable al final de cada uno, priorizar por valor para el cliente, usar equipos chicos, ser flexible y detectar y eliminar obstáculos. Tres ideas lo sostienen:

- **Time boxing**: cada actividad tiene un tiempo fijo y no negociable; se trabaja ese tiempo y se evalúa.
- **Desarrollo iterativo**: el trabajo se organiza en ciclos que se repiten (sprints), refinando lo hecho.
- **Desarrollo incremental**: el producto crece con piezas completas y funcionales. (La Mona Lisa: iterativo es bocetar todo y refinar; incremental es terminar la cara, luego una mano... Scrum combina ambos.)

Rol	**Responsabilidades**
Product Owner (PO)	La voz del negocio. Define las características del producto y decide fechas y contenido de cada entrega; consolida lo que piden los *stakeholders* en una visión priorizada; vela por la rentabilidad (ROI); prioriza por valor; administra los cambios de prioridad sprint a sprint; **acepta o rechaza** los entregables. Asigna el *business value*.
Scrum Master	Facilitador, no jefe. Mejora la productividad del equipo y su autonomía; fomenta la cooperación; **protege al equipo de interferencias y elimina impedimentos**; asegura que se respete el proceso (reuniones, timeboxes); mejora prácticas y herramientas.
Development Team	Convierte requerimientos en software funcionando; acuerda con el PO el alcance de cada iteración (primero el *qué*, después el *cómo*); organiza su propio trabajo; muestra resultados. Es **multidisciplinario** (tiene todas las especialidades) y **autoorganizado** (nadie de afuera le asigna tareas). Estima los *story points*.

Artefactos, como un flujo:

- **PBI** (*Product Backlog Item*): unidad de trabajo: una funcionalidad, un defecto, un refactor o una tarea técnica.
- **Product Backlog**: todo lo que podría hacerse, **priorizado** (un único ítem en el tope). Los de arriba están más detallados; los de abajo, más gruesos. La parte no acordada aún es el *uncommitted backlog*.
- **Sprint Backlog**: los PBIs que el equipo se comprometió a completar en el sprint (el *qué*); las tareas en que se descompone cada uno son el *cómo*.
- **Sprint**: el timebox (1 a 4 semanas). **El alcance acordado no cambia**: no se agregan ni quitan ítems; lo que no se termina vuelve al Product Backlog.

ID	Item	Prio	Value	Effort	Status
1	As a buyer, I want to place a book in a shopping cart - Details	1	7	5	Done
2	As a buyer, I want to remove a book from the shopping cart - Details	2	5	2	Done
5	Improve transaction performance - Details	3	4	13	In Sprint
7	Investigate solutions for speeding up credit card validation	4	4	20	In Sprint
6	Upgrade all servers to Apache 2.2.3	5	2	13	Ready
8	Diagnose and fix order processing script errors (JIRA SC-234)	6	3	3	Ready
3	As a shopper, I want to create and save a wish list - Details	7	2	40	Not Ready
4	As a shopper, I want to add items to my wish list - Details	8	2	20	Not Ready

Figura 2: El ciclo Scrum (cátedra).

Reuniones.

- **Sprint Planning**, en dos partes: *el qué* (PO, SM y equipo negocian el alcance: el PO dice qué importa, el equipo cuánto puede comprometer) y *el cómo* (el equipo descompone cada ítem en tareas estimadas en horas).
- **Backlog Refinement** (*grooming*): se estiman, dividen, aclaran y repriorizan los ítems para el próximo sprint.
- **Daily Scrum**: 15 minutos, frente al tablero; cada uno responde ¿qué hice?, ¿qué voy a hacer?, ¿qué problemas tengo? No es un reporte al Scrum Master; los problemas se resuelven después.
- **Sprint Review**: se muestra **software funcionando** (no diapositivas) a PO y stakeholders; los cambios que surgen van al backlog.
- **Sprint Retrospective**: el equipo revisa su *proceso* (no el producto): qué salió bien, qué mejorar, qué acciones tomar. Las acciones deben ser **SMART**: específicas, medibles, alcanzables, relevantes y acotadas en el tiempo.

2.3 Historias de usuario, estimación y velocity

Historia de usuario (US) e INVEST

"Como [**rol**], quiero [**objetivo**], para [**beneficio**]". Ejemplo: "Como comprador, quiero agregar libros al carrito para elegir los que voy a comprar". Una buena historia es **INVEST**: **I**ndependiente (se puede reordenar), **N**egociable (no es un contrato cerrado), **V**aliosa (para el usuario o quien paga), **E**stimable, **S**mall (menos de 3 semanas-persona) y **T**esteable.

Los **criterios de aceptación** hacen testeable a la historia: son condiciones verificables que suelen combinar (a) lo que el usuario *puede hacer* ("puede buscar por autor o título"), (b) las *reglas* según el caso ("si el libro ya está en el carrito, se aumenta la cantidad; si no, se agrega con cantidad uno") y (c) la *confirmación visible* ("luego se muestra el carrito").

Historia de usuario o ítem técnico. Pregunta guía: ¿lo podría haber pedido un usuario sin conocimientos técnicos porque necesita hacer algo nuevo? Si sí, es una historia. "Mejorar la performance de las transacciones", "investigar soluciones para validar tarjetas", "actualizar los servidores a Apache 2.2" o "corregir el script de compras" están en infinitivo, no tienen rol y benefician al sistema: son **ítems técnicos** del equipo.

Business value, story points y velocity. El **business value** (lo aporta el PO) mide el beneficio para el cliente. Los **story points** (los estima el equipo) miden el *esfuerzo relativo*: en vez de "¿cuántas horas?" se pregunta "¿es más grande o más chica que esta otra?", con una escala no lineal tipo Fibonacci (1, 2, 3, 5, 8, 13, ∞; el infinito indica que hay que partir la historia). La **velocity** es la cantidad de story points que el equipo completa por sprint: funciona como un presupuesto. Al principio se estima; con varios sprints se toma el promedio real. El **burndown chart** muestra el esfuerzo restante por día: si la línea real queda *por encima* de la ideal, el sprint va atrasado; si queda plana, el trabajo no avanza.

Procedimiento de selección para el sprint (TP0)

#	Ítem	V	E	V/E
2	sacar libro del carrito	5	2	2,50
1	poner libro en carrito	7	5	1,40
6	corregir script compras	3	3	1,00
8	agregar a favoritos	4	5	0,80
3	mejorar performance	4	13	0,31
4	investigar tarjetas	4	13	0,31
7	lista de favoritos	2	8	0,25
5	upgrade Apache	2	13	0,15

Con velocity 20: (1) calcular valor/esfuerzo; (2) ordenar de mayor a menor; (3) tomar en orden acumulando puntos sin pasarse: 2 $\rightarrow$ 2, 1 $\rightarrow$ 7, 6 $\rightarrow$ 10, 8 $\rightarrow$ 15; los siguientes (13, 13, 8, 13) no entran; (4) verificar que ningún ítem más chico quepa en los 5 restantes. **Sprint: #2, #1, #6 y #8 (15 puntos).** El ratio *orienta*: el PO puede priorizar otra cosa por dependencias, riesgos o fechas comprometidas.

Planning Poker. Técnica de estimación ágil (artículo recomendado por la cátedra): cada miembro del equipo que va a construir recibe un mazo (0, 1, 2, 3, 5, 8, 13, 20, 40, 100, "?", ∞); el PO presenta la historia y aclara dudas; todos eligen una carta **en secreto** y la muestran **a la vez**; si no coinciden, el que puso la más alta y el que puso la más baja explican sus razones y se vuelve a votar hasta el consenso. La revelación simultánea evita el **anclaje** (que la primera opinión, a menos que sea la del jefe, arrastre a todos), y la discusión de los extremos saca a la luz información que solo tenía uno. Es un *Delphi ágil*. Fibonacci se usa porque la distancia entre valores crece con la incertidumbre: discutir entre 37 y 38 sería precisión falsa.

En resumen Ágil son valores (4) y principios (12); Scrum es un framework con 3 roles (PO, SM, equipo), artefactos (backlogs, sprint con alcance congelado) y 5 reuniones. Las historias siguen el formato como/quiero/para, cumplen INVEST y llevan criterios de aceptación. El PO da el valor, el equipo los story points; la velocity limita lo que entra en el sprint y el ratio valor/esfuerzo orienta el orden.

3 Proyectos: definición, parámetros, clasificación y etapas [TP1 §1–7]

3.1 Qué es un proyecto y qué es administrarlo

Proyecto

Una **secuencia de actividades** con un **propósito**, que debe completarse en un **tiempo** determinado, dentro de un **presupuesto** y de acuerdo a una **especificación**. Dicho corto: cualquier actividad que dé como resultado un producto o entregable.

Cada palabra tiene consecuencias: las actividades son **únicas** (nunca se hicieron igual; eso separa un proyecto de una operación rutinaria), **complejas**, **conectadas** (hay dependencias), con un **propósito simple** y bajo **tiempo, presupuesto y especificación** (que puede variar). Todo proyecto **comienza con el enunciado de un problema**. Se evalúa desde cuatro preguntas: ¿resuelve el problema del usuario?, ¿el usuario está satisfecho con el *proceso*?, ¿lo está la gerencia?, ¿lo está el equipo? El ambiente actual es de *alta velocidad, alto cambio, bajos costos, alta complejidad*

y alta incertidumbre, y exige flexibilidad, comprensión del negocio y hacerse cargo del proyecto.

Administración de proyectos (PMI): aplicar conocimientos, habilidades, herramientas y técnicas a las actividades de un proyecto para satisfacer sus requerimientos. En software, combinar administración e ingeniería para entregar con **mínimo costo**, **mínimo tiempo** y **buena calidad**. Los datos que da la cátedra: menos de un tercio de los proyectos termina en tiempo y presupuesto, pero con una cultura de gestión fuerte el 71 % cumple sus objetivos. Desafíos: innovación, complejidad, requerimientos ambiguos, falta de competencias, herramientas inmaduras, regulaciones, plazos, proveedores, retener personal, equipos distribuidos y multiculturales.

3.2 Parámetros, triángulo de alcance y corrimientos

Todo proyecto se mueve dentro de **cinco parámetros**, que también son sus restricciones: **ámbito** (alcance: qué se hace y qué no), **calidad**, **costo**, **tiempo** y **recursos** (personas, equipos, licencias). No son independientes: el **triángulo de alcance** tiene vértices *tiempo*, *costo* y *alcance y calidad*, y mover uno obliga a mover otro. Si el cliente pide más funcionalidad con la misma fecha, sube el costo (más recursos) o baja la calidad. Sirve para **escalar problemas** (un atraso no se arregla "por arte de magia") y como **base para definir el impacto de un cambio**. El proyecto es un sistema dinámico que el LP mantiene en equilibrio.



Figura 3: Triángulo de alcance (cátedra).

Quién controla qué: el **líder de proyecto** controla la utilización de los recursos y el cronograma; el **gerente**, los niveles de recursos y los costos; el **cliente**, el alcance, la calidad y las fechas de entrega. Si el cliente pide un cambio, decide él, pero el LP traduce el impacto y el gerente decide si pone recursos.

Corrimientos (desvíos progresivos y silenciosos de un parámetro):

- **De alcance** (*scope creep*): todo cambio que no estaba en el plan. El LP debe acomodar el proyecto (con el proceso de cambios), no ignorarlo.
- **De esperanza** (*hope creep*): reportar "sin demoras" esperando recuperar antes del próximo informe. Se combate verificando los informes, por ejemplo con **controles aleatorios**.
- **De esfuerzo** (*effort creep*): se trabaja y se reporta avance, pero la tarea nunca termina.
- **De rasgos** (*feature creep*): el equipo agrega características que nadie pidió; cada una necesita pruebas, datos, documentación y capacitación, y consume tiempo y costo sin autorización.

3.3 Éxito y fracaso

Un proyecto puede fracasar **al comienzo** (no se planificó: avanza rápido hasta el 90 % y se queda ahí), **durante el desarrollo** (análisis y diseño mal documentados, responsabilidades confusas, falta de planificación de recursos, codificación prematura, sin revisiones gerenciales ni *walk-throughs*, sin estándares, deserciones, "fuerza bruta" de horas extra) o **al final** (pocas pruebas, mala performance, mantenimiento caro, cancelación por fondos, contexto, gente clave o desastre técnico). Para la cátedra, planificar es saber **a dónde** se va, **cómo** se llega y **cómo se prueba** que se llegó. Los proyectos **exitosos** tienen un comienzo claro con plan detallado, criterios de medición aceptables, monitoreo cercano, documentación clara y revisiones frecuentes, procedimientos para detectar problemas y un costo razonablemente cercano al estimado.

3.4 Clasificación de proyectos (A–D)

Cada proyecto es único, pero no se puede inventar un proceso para cada uno: se clasifican para aplicar **políticas por grupo**. Criterios: riesgo, valor para el negocio, duración, complejidad, tecnologías (conocidas a nunca usadas), departamentos afectados y costo.

Tipo	Duración	Riesgo	Complejidad	Tecnología	Complicaciones	Planificar
A	>18 meses	alto	alta	de avanzada	seguras	5 días o más
B	9–18 meses	medio	media	actual	alta probabilidad	3–4 días
C	3–9 meses	bajo	baja	buena	alguna probabilidad	hasta 2 días
D	<3 meses	muy bajo	muy baja	práctica	ninguna	menos de 1 día

A: alto valor y alta complejidad, merecen todos los recursos (una tecnología nueva en un producto clave). **B**: más cortos pero significativos y tecnológicamente desafiantes. **C**: los más frecuentes; unas 5 personas hasta 6 meses, con recursos no dedicados. **D**: apenas son proyectos (cambios menores en un proceso). No todos los procesos son obligatorios: **el POS es requerido en los cuatro tipos**, igual que la aprobación del pedido, la propuesta y su aprobación, el cronograma, el reporte de estado, la aprobación de entregables y la auditoría post-implementación. Las condiciones de satisfacción, la sesión de planificación, el kick-off, las reuniones de equipo y las notas del proyecto son *opcionales* en C y D.

3.5 Etapas, hitos y ciclos de vida

Para controlar algo hay que descomponerlo: en partes del producto, en trabajo de personas y en **fases de tiempo**. Cada fase produce uno o más **hitos**: eventos que prueban claramente que una etapa concluyó satisfactoriamente (entrega del documento de requerimientos, de la especificación, del diseño, del código, el deploy). Los cinco procesos típicos son:

- **Concepción** (*scoping*): define **qué** se hará, incluidos los criterios de éxito: reclutar al LP, obtener y documentar las reales necesidades, negociar cómo se cubrirán, escribir el POS y ganar la aprobación para planificar.
- **Planificación**: define **cómo**: definir el trabajo, estimar tiempo, recursos y costo, secuenciar, cronograma inicial, **plan de contención de riesgos**, documentar y ganar la aprobación para lanzar.
- **Lanzamiento**: reclutar el equipo, emitir la descripción del proyecto, reglas del equipo, **proceso de cambio de alcance**, comunicaciones, terminar el cronograma, paquetes de trabajo.
- **Monitoreo y control**: medir la performance, reportes de estado, monitorear riesgos, procesar pedidos de cambio, descubrir y resolver problemas.
- **Cierre**: aprobación del cliente, instalación de entregables, reporte final, auditoría post-implementación.

Las combinaciones dan los **ciclos de vida**: *concepción–planificación–lanzamiento* (una o dos personas: sirve como lista de tareas); *más control* (lanzar es solo el 50 %: el control anticipa problemas); *las cinco etapas* (el LP aprende de la experiencia del proyecto terminado).

3.6 Atributos críticos

Atributo crítico

Una calidad o un recurso que puede hacer **colapsar el sistema** si supera ciertos límites (su **peor caso aceptable**). Se especifica con niveles medibles, no con una frase vaga.

Capacidad de trabajo (cátedra)

Debe alcanzar para el trabajo diario normal. *Peor caso*: 4 s promedio por transacción. *Nivel planificado*: menos de 1 s (0,2 s si hay más de 100.000 transacciones diarias). *Mejor caso*: 0,1 s. En el caso real se “presumió” que habría capacidad suficiente, y como nunca se escribió como requerimiento, nadie la verificó.

Principios de administración: especificar claramente todos los atributos críticos; satisfacerlos *simultáneamente*; en términos testeables y medibles, identificando el peor caso; **nunca** entregar

sistemas grandes y complejos de una sola vez (*big-bang*); no creer ciegamente en ningún método; "si no se sabe lo que se está haciendo, no hacerlo en gran escala".

En resumen Proyecto: actividades únicas, complejas y conectadas, con propósito, tiempo, presupuesto y especificación. Cinco parámetros en tensión (triángulo tiempo–costo–alcance/calidad); el LP controla recursos y cronograma, el gerente niveles de recursos y costos, el cliente alcance, calidad y fechas. Cuatro corrimientos. Tipos A a D; el POS siempre es obligatorio. Cinco etapas con hitos. Atributos críticos con peor caso medible.

4 Concepción: COS, POS y aprobación [TP1 §8, TP2 §1–3 y §6]

4.1 La falla de comunicación cliente–proveedor

La causa más común de fracaso no es técnica sino **comunicacional**: se construye algo distinto de lo que el cliente necesitaba. Los clientes esperan más de lo que los proveedores están preparados para entregar, y hay **dos supuestos encadenados que nadie verifica**: que el LP entendió lo que pidió el *requiriente* (el cliente), y que lo que el cliente pidió es lo que realmente necesita. Por eso el LP pregunta **por qué** lo quiere, no solo qué quiere. **Regla de oro**: nunca comenzar un proyecto sin estar seguro de que la solución propuesta es lo que realmente satisface al cliente. Las herramientas de la concepción (se eligen según el proyecto) son: COS, POS, reuniones de definición, especificación de requerimientos, diagramación de procesos de negocio, prototipos, validación de casos de uso, administración de la provisión, outsourcing y aprobación para la planificación.

4.2 Condiciones de Satisfacción (COS)

COS

Una **conversación estructurada** (un proceso, no un documento) entre el requiriente y el probable LP para llegar a un entendimiento compartido de qué se va a hacer. Su producto tangible es el POS.

Tiene **cuatro etapas cíclicas**:

1. **Requerimiento**: el cliente pide, en el lenguaje de su negocio.
2. **Clarificación**: el proveedor explica con sus palabras lo que entendió, *en el lenguaje del requiriente*.
3. **Respuesta**: el proveedor dice qué es capaz de proveer (lo pedido, una parte o una alternativa).
4. **Acuerdo**: el requiriente explica con sus palabras lo que entendió que se le ofrece.

Obligar a ambos a *parafrasear*, en lugar de asentir, hace visibles los malentendidos cuando corregirlos cuesta una charla. Lo normal es que no haya acuerdo en la primera vuelta. Datos operativos: el entregable es el POS de **una página**, firmado por ambas partes; la concepción termina cuando lo aprueba un **gerente senior**; el COS sirve mejor en proyectos pequeños y medianos; es **dinámico**: se especifica qué productos demostrarán que se cumplió (criterios de éxito) y se revisa durante el monitoreo.

4.3 El POS (Project Overview Statement)

POS — Enunciado General del Proyecto

Documento breve de la concepción que dice **qué** se va a hacer, **por qué** y **qué valor comercial** tendrá. Esquema de las diapositivas: nombre y número del proyecto, líderes, **problema/oportunidad**, **metas**, **objetivos**, **criterios de éxito**, **suposiciones, riesgos y obstáculos**, preparado por/fecha, aprobado por/fecha.

Funciones organizacionales: base para **aprobar y autorizar recursos**; insumo para **priorizar la**

cartera de proyectos (casi siempre hay más ideas que recursos); **referencia para la planificación**; **solicitud** formal de la iniciativa. Cada campo responde una pregunta distinta:

Campo	Qué debe decir	Ejemplo
Problema / oportunidad	la situación actual insatisfactoria o la ventaja no aprovechada, *sin proponer la solución ni comprometer métricas*	"El instituto gestiona los alumnos con planillas Excel; esto produce una distribución inadecuada de la información y servicios ineficientes."
Meta	la finalidad del proyecto; cualitativa, corta, sin fechas necesariamente; verbo como facilitar, lograr, mejorar; debe hacer que la gerencia quiera seguir leyendo	"Facilitar el acceso de los usuarios a sus datos personales para que puedan modificarlos."
Objetivos	el desglose de la meta: **medibles y con plazo**; conjunto **necesario** (todos aportan) y **suficiente** (entre todos alcanzan)	"El 80 % de los usuarios accede a sus datos a menos de una semana de la puesta en producción."
Criterios de éxito	métricas cuantitativas que prueban que el proyecto cumplió lo prometido; pueden mirar al equipo o la organización	"Satisfacción de los usuarios de al menos 4 sobre 5."
Suposiciones, riesgos y obstáculos	lo que se da por sentado o podría hacer fracasar o demorar el proyecto (conecta el POS con el análisis de riesgos)	"Se supone que la prepaga acepta renegociar; los datos de salud son sensibles."

Objetivos bien escritos: las cinco características (específicos, medibles, *asignables* a un responsable, realistas y limitados en el tiempo) y los cuatro componentes (una *salida*, un *marco de tiempo*, una *medida* y una *acción*). Si en esta etapa se dan estimaciones, hay que aclarar que son *preliminares*.

FCE (Factores Críticos de Éxito) vs. criterios de éxito

Un **FCE** responde "¿qué debería sucedernos a nosotros y al cliente para decir que el proyecto fue un éxito?". Son pocos, decisivos, medibles y en términos de negocio; sirven para "vender" el proyecto y que la gerencia autorice recursos. Los **criterios de éxito** son su operacionalización: las métricas que verifican que el FCE se cumplió. Ej.: FCE "que el área de ventas adopte el sistema y abandone el proceso manual"; criterio "el 90 % de las órdenes del trimestre se carga por el sistema".

4.4 Aprobación para la planificación

Terminado el POS, se envía con sus adjuntos a los gerentes senior. No es un trámite: evalúan si el proyecto tiene valor y está bien definido. **Aprobar el POS es asignar recursos para planificar en detalle**; la aprobación del proyecto para ejecutarlo viene después, con el plan. Puede haber varias iteraciones. Cumple una función para tres audiencias: a la *alta gerencia* le justifica dedicar recursos; al *cliente* le muestra que el proyecto está bien definido; al *equipo* le asegura que gerencia y cliente están alineados. Participan el LP, los gerentes de los recursos (de usuarios), los gerentes de los procesos que intercambian información con el sistema, el cliente y la alta gerencia. Los gerentes suelen preguntar por la importancia del problema, la relación con los FCE, si la meta responde al problema, si los objetivos representan la meta, si el valor de negocio justifica el gasto y si se pueden mitigar los riesgos. **Resultados posibles**: **aprobado**, **rechazado**, **reformulación** (se reenvía), **postergado** o **derivado a un portafolio**.

En resumen El COS es una conversación en cuatro etapas (requerimiento, clarificación, respuesta, acuerdo) cuyo resultado es el POS: una página, firmada, con problema (sin solución), meta (cualitativa), objetivos (medibles, con plazo, necesarios y suficientes), criterios de éxito y riesgos. Los FCE venden el proyecto; los criterios los miden. La aprobación del POS habilita a planificar, no a ejecutar.

5 Requerimientos y soluciones [TP2: apunte de Requerimientos y §4]

5.1 Requerimientos: definición, proceso y tipos

Requerimiento

Equivalentemente: (1) una función que el sistema debe realizar; (2) una característica deseada del sistema; (3) un enunciado sobre el sistema que los *stakeholders* acuerdan que debe satisfacerse para que el problema del cliente quede adecuadamente resuelto.

La tercera definición es la importante para la gestión: un requerimiento nace de un *acuerdo* y existe para *resolver un problema*. Un **stakeholder** (interesado) es cualquier persona u organización cuyos intereses se ven afectados por el proyecto (cliente, usuarios, gerencia, equipo, soporte, reguladores).

El proceso de requerimientos tiene cinco actividades, **no lineales** (se realimentan): **elicitación** (descubrir los requerimientos con los stakeholders), **especificación** (documentarlos de forma clara y sin ambigüedad; el SRS), **modelamiento y análisis** (representarlos con modelos y detectar faltantes, contradicciones o imposibles), **validación** (confirmar con los stakeholders que son los requerimientos *correctos*) y **administración** (gestionar cambios y trazabilidad durante todo el proyecto).

	Requerimiento funcional	**Requerimiento no funcional**
Definición	Describe la interacción entre el sistema y su entorno: cómo se comporta ante un estímulo.	Describe restricciones que **limitan la selección de soluciones**: con qué calidad o bajo qué límites.
Cumplimiento	**Presente o ausente** en un plan o sistema (binario).	Tiene una **escala de demanda** y admite negociación según las prioridades.
Se gestiona como	lista de ítems (está o no está en el alcance).	atributo con escala, test y niveles.
Ejemplo	"El sistema permite registrar un nuevo usuario."	"El registro se completa en menos de 2 s."

Tipos usados para organizar el documento: funcional, de interfaz (con otros sistemas o dispositivos), de datos, de ingeniería humana (usabilidad), operacionales (disponibilidad, backups, soporte), restricciones de diseño ("debe usar tal lenguaje"), de seguridad, etc.

Cualidades de una buena especificación: **correcta** (refleja una necesidad real), **consistente** (sin contradicciones), **factible**, **verificable** (hay forma objetiva de comprobarla), **completa** y **trazable** (hacia su origen y hacia el diseño, código y pruebas). **Estándares** que listan las diapositivas: Guía IEEE P1233/D3, Guía IEEE Std. 1233, IEEE Std. 830-1998, ISO/IEC 12119-1994 e IEEE Std. 1362-1998 (ConOps).

5.2 Funcionales: separar función, calidad y solución

Lo esencial es identificar las **funciones** del sistema (algo que el sistema hace). Suelen registrarse como una lista que completan los usuarios con ayuda del analista. Dos errores clásicos:

- **Mezclar función y calidad**: "redactar un manual de usuario *legible*". "Redactar un manual" es funcional (está o no está); "legible" es un atributo (escala). Hay que separarlos para poder verificar cada parte.
- **Soluciones disfrazadas de requerimientos**: lo que el usuario enuncia en la elicitación suele ser su forma de decir lo que desea, no la mejor solución. Si se registra como requerimiento, se

cierra la puerta a alternativas. Técnica: preguntar **¿por qué?** o **¿para qué?** hasta llegar a la necesidad real. "El usuario requiere construir estándares" → ¿para qué? → para facilitar el mantenimiento y posibilitar la rotación de personal.

El ejercicio del apunte

"Nosotros, la organización XX, queremos escribir un manual de desarrollo de sistemas para mejorar sustancialmente la productividad del personal." **No funcional**: mejorar sustancialmente la productividad (se mide en una escala; es el objetivo real). **Funcional**: escribir el manual (está o no está). **Solución**: el manual, elegido de antemano; hay que cuestionarlo, porque quizás una capacitación o mejores herramientas mejoran más la productividad.

Ficha de un requerimiento funcional (ejemplo de las slides)

req. id / categoría	F44 / Funcional
descripción	Aplicantes a beneficios, empleados y autoridades deben poder registrarse; se crea una cuenta para autenticación y personalización. Los empleados los crea el administrador; los aplicantes se registran solos en el portal.
términos	Cuenta, Aplicante, Autenticación, Registración, Usuario (glosario para que todos entiendan lo mismo).
justificación / prioridad	Esencial para autenticar y personalizar / Alta.
dependencias / documentos	F43 / —
factibilidad	Existen soluciones maduras (servicios de directorio).
verificación	Al registrar un aplicante sin cuenta, se crea una cuenta en el sistema.

Cada campo fuerza una cualidad: id y dependencias → trazabilidad; justificación → correcta; factibilidad → factible; verificación → verificable.

5.3 No funcionales: atributos, escala, test y niveles

Atributo

Un **concepto de calidad o de recurso** que describe el sistema **cuantitativamente**; representa restricciones y se define para cada proyecto. Los atributos de calidad constituyen los requerimientos no funcionales.

- La **especificación de atributos** es una lista estructurada, con historial de cambios; los atributos críticos se controlan durante todo el ciclo de vida. Todos deberían volverse medibles: la gente falla al definir escalas porque culturalmente no se exige.
- Se organizan en **jerarquías**: "mejor calidad y organización más productiva" → satisfacción de clientes, productividad de sistemas, adaptabilidad de los empleados. Se descompone hasta el nivel que permita controlar. Si son más de 7 a 10, se agrupan bajo un nombre corto ("Performance del sistema": carga semanal, disponibilidad, tiempo de respuesta).
- Conviene identificar primero los de **bajo nivel** (concretos, visibles) y subir preguntando *por qué*: el de alto nivel muestra otras soluciones (el usuario pide "confiabilidad" y en realidad quiere *disponibilidad*, que también depende de mantenibilidad e integridad).
- Se especifican **al principio**, antes de cualquier solución, en lenguaje que entiendan todos (la prueba es que la gerencia usuaria firme), y **no se congelan**. Las soluciones se juzgan contra *todos* los atributos juntos.
- Si cuesta definir una medida, el atributo es de alto nivel: "amigable" → aprendizaje (horas para aprender, planificado 5) y productividad (tareas por hora, planificado 20).

Campo	Significado	Ejemplo: tiempo de actualización
Escala	unidad de medida, *qué* se mide (voltios)	ms desde el evento hasta la BD actualizada
Test	procedimiento, *cómo* se mide (voltímetro); mejor si ya está en uso	un día aleatorio, mínimo 10 casos, ideal 50
Peor caso	peor nivel aceptable bajo cualquier circunstancia: por debajo hay **fracaso**; base de un contrato	180.000 ms
Nivel planificado	lo que el proyecto **se compromete** a lograr (éxito formal); realista pero ambicioso, no la perfección	100 ms
Mejor caso	lo mejor alcanzado en cualquier circunstancia: **solo informativo**, "límite de la ingeniería"	1 ms
Nivel actual	valor en un sistema existente (citar la fuente); planificado − actual = mejora prometida	sin medida

De peor a mejor: peor caso → (nivel actual) → nivel planificado → mejor caso. Para fijar el peor caso: imaginar un nivel inaceptable e irlo mejorando hasta que algún usuario lo acepte en alguna circunstancia.

5.4 Soluciones

El ejemplo de la **hamaca** (lo que pidió el cliente, lo que entendió el vendedor, lo que diseñó el equipo...) muestra que la función ("una hamaca colgada de un árbol") era clara; lo que falló fue la especificación de **atributos**: la calidad y los recursos de cada etapa.

Solución

Un **conjunto de ideas cuya implementación impacta positivamente en al menos una parte del problema**. Una idea sin efectos positivos en los requerimientos no es una solución. Es válida *en la medida en que mejor sirva a los requerimientos actuales*: si estos cambian, puede dejar de serlo.

- Hay que tener soluciones suficientes para **todos los objetivos del POS**. Se empieza por las funciones (más visibles). Cadena: *las funciones determinan los atributos y los atributos determinan las soluciones.*
- Para evaluar una solución hace falta: requerimientos especificados, sus contribuciones positivas, sus efectos colaterales negativos y su consumo de recursos frente al presupuesto.
- Requerimiento funcional = un estado futuro verdadero o falso; no funcional = escala; solución = una idea que impacta en ellos. **Que una solución no se transforme en un requerimiento.**
- **Flexibilidad**: una solución puede especificarse como *requerimiento funcional* (solo si es más prioritaria que el atributo que perjudica), como solución *para un rango* de un atributo (para evitar extremos) o como **non-sancta** (sujeta a cambios: el caso por defecto).
- Los **gerentes** enuncian sus problemas en términos de objetivos; los **problem solvers** encuentran las soluciones. Son procesos separados.
- **Mapeo**: varias soluciones pueden impactar en un atributo, una solución en varios, y algún atributo puede no tener solución (hay que revisar si el objetivo es alcanzable).
- **Estimación del impacto**: cuánto contribuye cada solución a cada atributo, expresado como estimación o *rango* (incertidumbre honesta), en una **tabla de estimación de impacto**. Beneficios: permite discutir con el decisor, entrena a estimar, motiva a cumplir, habilita la discusión profesional y aclara objetivos mal expresados.
- **Comparación**: técnica tabular que asegura consistencia y comunicación. Se **suman alge-**

braicamente los impactos: en calidad, más es mejor; en recursos, menos es mejor.

$$\text{Razón de impactos} = \frac{\sum \text{impactos de calidad}}{\sum \text{impactos de recursos}} \quad \text{(mayor es mejor)}$$

- **Proceso de 7 pasos**: (1) un atributo por vez, lluvia de ideas; (2) estimar efectos colaterales; (3) **eliminar las que producen el peor caso** en algún atributo; (4) repetir; (5) si se cubren todos los niveles planificados, ir al 7; (6) si no, buscar más experiencia o negociar los niveles; (7) testear las hipótesis (inspección de Fagan, entregas parciales y medición).
- **Especificar soluciones**: resumidas, precisas, cada idea nueva en su propio enunciado, con una **etiqueta** única (numérica o mnemotécnica, que no envejece; jerárquica; temporal para versiones), útil para tablas y gestión de configuración.

Comparación de soluciones (TP2, ejercicio 5)

	S1	S2	S3
A	20	30	10
B	80	10	50
C	5	20	50
D	15	20	50
Suma	120	80	**160**
Costo	40 %	40 %	70 %
Razón	**3,00**	2,00	2,29

Cada celda es el % del nivel planificado del atributo que aporta la solución. (a) Por impacto se elige **S3** (160), que además es la más pareja (S1 deja C en 5 %, casi el peor caso). (b) Atributo **determinante**: se excluye uno por vez y se recalcula; sin C, S1 suma 115 y S3 110: la preferencia cambia, así que **C** es determinante. (c) Con costos, la razón favorece a **S1** (3,00). (d) El líder de S2 puede *reconsiderar la implementación* (siempre válido: bajar costo o mejorar B) o la *relevancia de los atributos* (solo si se acuerda con quien fija los objetivos, no para hacer ganar a S2).

Inspección de Fagan: revisión formal de *documentación* por **colegas de todos los niveles salvo el gerente** (para que la crítica sea honesta), en **puntos prefijados**, buscando **todo tipo de defecto**, con pasos predefinidos (preparación individual, reunión, reconstrucción de errores), reuniones de **hasta 2 horas**, un **moderador entrenado**, **roles** asignados y **checklists** de preguntas.

En resumen Funcional = presente/ausente; no funcional = escala negociable. Se separa función de calidad y se pregunta "¿por qué?" ante soluciones disfrazadas. Un atributo se especifica con escala, test, peor caso, nivel planificado, mejor caso y nivel actual. Las soluciones se comparan sumando impactos y con la razón calidad/ recursos; se eliminan las que producen el peor caso.

6 Gestión de riesgos [TP1 §9, TP2 §5]

6.1 Qué es un riesgo y cómo se clasifica

Riesgo y gestión del riesgo

Un riesgo es un **problema potencial**: puede ocurrir o no. Implica **incertidumbre** (si fuera seguro, sería una restricción: no existe un riesgo 100 % probable) y **pérdida** (consecuencias, impacto). El análisis y la gestión del riesgo son pasos para comprender y gestionar la incertidumbre: (1) **identificar**, (2) evaluar la **probabilidad**, (3) estimar el **impacto**, (4) elaborar el **plan RSGR**. Los planes de contingencia deben ser *realistas*.

Estrategia reactiva ("escuela Indiana Jones": "no te preocupes, pensaré en algo"; a lo sumo supervisa) vs. **proactiva** (la recomendada: empieza *antes* del trabajo técnico, identifica, valora y prioriza, y establece un plan). "Si no atacamos activamente a los riesgos, ellos nos atacarán."

Principios: el profesional *comparte* el conocimiento de los riesgos con colegas y clientes; la prevención es más efectiva en costo que la detección; al indicar desvíos se dan causas y acciones; el riesgo nunca se oculta a los *decision-makers*; si no se busca información de riesgo, se buscan problemas.

Clasificación	**Categorías**
Pressman	**Del proyecto**: amenazan el plan (presupuesto, calendario, personal, cliente, requisitos, tamaño, recursos). **Técnicos**: el problema es más difícil de lo que se creía (diseño, implementación, interfaces, mantenimiento, tecnología incierta). **Del negocio**: amenazan la viabilidad; subtipos de *mercado* (nadie lo quiere), *estratégico* (no encaja en la estrategia), de *ventas* (no se sabe venderlo), de *dirección* (se pierde el apoyo gerencial), de *presupuesto* (se pierden fondos o personal).
Genérico / específico	Genéricos: amenazan a todo proyecto. Específicos: propios de este proyecto; solo los identifica gente con experiencia en su tecnología, personal y entorno.
Charette	**Conocidos** (del análisis del proyecto, su entorno y su plan), **predecibles** (se extrapolan de otros proyectos), **impredecibles** (muy difíciles de anticipar).
Componentes (qué amenazan)	Riesgo de **rendimiento** (que el producto no cumpla), de **costo** (presupuesto), de **soporte** (poder corregirlo y adaptarlo) y de **planificación temporal** (entregar a tiempo). El impacto de cada controlador: despreciable, marginal, crítico, catastrófico.

6.2 Herramientas para identificarlos

La **identificación** es un intento *sistemático* de especificar las amenazas al plan. Dos herramientas, que se combinan (las categorías de Pressman son las carpetas; los conductores de Wysocki y los riesgos específicos del enunciado, lo que va adentro):

Lista de comprobación de elementos de riesgo (Pressman)	
Tamaño del producto	tamaño en LDC o PF y confianza en esa estimación; BD y transacciones; desvío respecto de proyectos similares; cantidad de usuarios; cambios previstos; cantidad de software reutilizado.
Impacto en el negocio	efecto en los ingresos; viabilidad según expertos; razonabilidad de la fecha; clientes con necesidades inconsistentes; interoperabilidad; sofisticación del usuario; documentación; regulaciones; costo del retraso y de un producto defectuoso.
Características del cliente	¿trabajamos antes con él?, ¿sabe lo que quiere y lo escribió?, ¿acepta reuniones?, ¿se comunica?, ¿participa de revisiones?, ¿deja trabajar?, ¿entiende el proceso de software?
Definición del proceso	¿la gerencia apoya un proceso estándar?, ¿hay proceso escrito?, ¿revisiones formales y documentadas?, ¿control de estándares y de cambios de requisitos?, ¿métodos y herramientas de análisis, prueba, planificación?, ¿métricas?
Entorno de desarrollo	¿hay herramientas de análisis, diseño y gestión?, ¿son apropiadas, integradas, con expertos y buena ayuda?
Tecnología a construir	¿es nueva para la organización?, ¿nuevos algoritmos o interfaces?, ¿hardware o software no probado?, ¿métodos no convencionales?, ¿el cliente duda de que sea factible?
Tamaño y experiencia de la plantilla	¿la mejor gente?, ¿suficiente y capacitada?, ¿asignada todo el proyecto?, ¿conoce las expectativas?, ¿habrá rotación?

Conductores de riesgo (*risk drivers*, Wysocki): condiciones o situaciones que pueden afectar desfavorablemente el éxito: planificación demasiado ambiciosa; subestimación del presupuesto; obligaciones de contrato mal interpretadas; estimación de tamaño inadecuada; hardware nuevo o poco familiar; personal calificado no adecuado; plan de desarrollo inadecuado; métodos pobres de ingeniería; falta de soporte político o de necesidad real; análisis o administración de riesgos inadecuados; expectativas poco realistas; procesos o tecnologías poco familiares; modelo de desarrollo inadecuado; requerimientos definidos pobremente; estructura organizacional inadecuada.

6.3 Evaluación: tabla de riesgos, línea de corte y matrices

La **proyección** del riesgo lo mide por su probabilidad y sus consecuencias: se define una escala de probabilidad, se definen las consecuencias, se estima el impacto y se anota la exactitud de la estimación. La **tabla de riesgos** (en una planilla) tiene las columnas *elemento de riesgo* (el riesgo concreto, no la categoría), *categoría*, *probabilidad* (%; cada integrante estima y se usa la media) e *impacto* (TP1: 1 catastrófico, 2 crítico, 3 marginal, 4 despreciable). Después:

1. **Justificar** cada probabilidad e impacto con datos del enunciado ("¡Justificar!", pide la cátedra).
2. **Ordenar** primero por probabilidad (de mayor a menor) y, a igualdad, por impacto (el más grave primero).
3. Trazar la **línea de corte**: un umbral justificado; los de arriba reciben un plan RSGR, los de abajo solo se monitorean.

Criterio de la teoría: *gran impacto y poca probabilidad* no demanda mucha gestión; *gran impacto y probabilidad moderada* o *impacto medio y probabilidad alta* sí. El impacto real depende de la **naturaleza** del problema, su **alcance** (severidad y distribución) y **cuándo** ocurre. La tabla se **reevalúa periódicamente**.

Escala de impacto: dos convenciones

El TP1 usa 1 = catastrófico y 4 = despreciable (el número bajo es el grave). El material del TP2 la enuncia al revés (y con un orden inconsistente). En el examen, usá la escala que te den y escribí siempre la **etiqueta** junto al número.

Resolución de la cátedra: sistema de salud para empleados (TP1, problema 1)

#	Riesgo (categoría)	Impacto	Prob.
1	Costo del proyecto demasiado alto en función de los resultados (negocio)	Crítico	75 %
7	Pérdida de personal (plantilla)	Crítico	70 %
9	Interfaces poco atractivas que no logren los objetivos de uso (tecnología)	Catastrófico	60 %
2	Fecha de entrega ajustada/comprometida (proceso)	Marginal	60 %
11	Insuficiente variedad de actividades para ofrecer (negocio)	Catastrófico	50 %
12	Baja aceptación de las propuestas por los empleados (negocio)	Catastrófico	50 %
3	Subestimación del presupuesto (tamaño)	Crítico	50 %
	— línea de corte: probabilidad $\geq 50\,\%$ —		
5, 6	Poca disponibilidad del cliente; cambios en los requerimientos	Crítico	40 %
10	Poca experiencia con las tecnologías	Crítico	20 %
4, 8	Problemas de comunicación con el cliente; pérdida de soporte político	Catastrófico	10 %

RSGR del riesgo 7 (pérdida de personal). *Reducción*: reuniones para minimizar lo que dispara las salidas (dinero, conflictos); identificar tareas críticas y prever personal de respaldo; énfasis en la documentación. *Supervisión*: seguimiento del equipo, de las asignaciones y de la documentación. *Gestión*: reasignar tareas (por eso importa documentar); horas extra o incorporar un recurso nuevo.

Matrices de riesgo: cruzan probabilidad e impacto para ver de un vistazo la severidad. La **numérica 4×4** multiplica probabilidad de amenaza (1 insignificante, 2 baja, 3 media, 4 alta) por magnitud del daño (misma escala): producto 1–6 bajo, 8–9 medio, 12–16 alto. La **cualitativa** cruza cinco niveles de probabilidad (raro, poco probable, posible, muy probable, casi seguro) con cinco de consecuencias (despreciables a catastróficas) y da riesgo bajo, medio, alto o muy alto.

6.4 El plan RSGR y su costo

Plan RSGR (Reducción, Supervisión y Gestión del Riesgo)

Para cada riesgo sobre la línea de corte: **Reducción** (*antes*: evitar que ocurra y mitigar sus efectos si ocurre), **Supervisión** (*durante*: controlar el riesgo, verificar que se ejecuten las acciones de reducción, detectar señales) y **Gestión** (*cuando ya ocurrió*: manejarlo y minimizar el impacto). El **PRSGR** es el documento que registra el análisis y acompaña al plan del proyecto; sus acciones son *evitar* el problema, *valorar* si los riesgos ocurren, *asegurar* que se cumplan los pasos y *recopilar información* para futuros proyectos.

Patrón de la cátedra: la reducción mejora información o planificación (refinar estimaciones, documentar, capacitar, encuestas, prototipos); la supervisión fija **puntos de control** y compara con lo esperado; la gestión **escala** (notificar a la gerencia, renegociar, reasignar, contratar).

Herramientas para desarrollar y aplicar los planes (pregunta del TP1): para desarrollarlos, la *lista de comprobación* y los *conductores* (identificar), la *tabla de riesgos* en una planilla (evaluar y ordenar), la *línea de corte* y las *matrices* (priorizar) y el documento *PRSGR*; para aplicarlos, puntos de control y montos de referencia en el cronograma, reuniones de avance, la bitácora de pendientes y la reevaluación periódica de la tabla, con una herramienta de planificación y seguimiento (MS Project, Jira, una planilla compartida) para registrar responsables y fechas.

El costo de gestionar riesgos: los pasos del PRSGR aumentan el costo del proyecto, y parte de la gestión es ver cuándo el costo supera al beneficio. En un proyecto grande hay 30 a 40 riesgos con 3 a 7 pasos cada uno: la gestión de riesgos se vuelve otro proyecto. Por eso se aplica la **regla de Pareto (80/20)**: pocas causas explican la mayoría de los efectos, y el esfuerzo se concentra en los riesgos sobre la línea de corte. **Expresar el riesgo**: importa la confianza en los números; se puede dar una desviación, varios niveles (peor, planificado, actual), una probabilidad o los factores que influyen. **Exposición al riesgo**: ER = probabilidad × costo; sirve para ajustar el presupuesto (un colchón proporcional) o para asegurar el riesgo.

En resumen Riesgo = incertidumbre + pérdida. Se clasifican por origen (proyecto, técnico, negocio; genérico o específico), por Charette y por componente. Se identifican con el checklist y los conductores, se evalúan en una tabla justificada (impacto 1 = catastrófico), se ordenan por probabilidad y luego impacto, se corta con criterio y se planifica con RSGR, concentrando el esfuerzo según Pareto.

7 Planificación [TP3]

7.1 Para qué planificar y la JPPS

Planificar es convertir un POS aprobado en un plan detallado que conteste "¿cuánto cuesta y cuándo lo tengo?". Para el LP el plan no es solo una hoja de ruta: es una **herramienta para decidir** (¿qué pasa si alguien se enferma o se adelanta la fecha?). Hay que **esperar el cambio**; el plan vale porque **reduce la incertidumbre**, **aumenta el conocimiento** y **mejora la eficiencia** ("los planes no sirven, pero planificar lo es todo"). Las herramientas (MS Project, ProjectLibre, Jira) permiten investigar alternativas, pero manejan mal a las personas (asignan a alguien al 300 %), tienen costos de licencia y aprendizaje, y solo son tan buenas como las **estimaciones** que se les cargan. El tiempo de planificación depende del tipo: D menos de un día, C hasta dos, B tres o cuatro, A cinco o más.

JPPS (Joint Project Planning Session)

Sesión grupal donde se genera el plan detallado. Objetivo: un plan que **satisfaga el COS** y **describa el POS**. Obstáculo: los participantes tienen poco tiempo (se usan reuniones virtuales). Lo construyen quienes lo van a cumplir: gana calidad (cada experto aporta) y compromiso.

Agenda: revisar el POS, crear el WBS, estimar duración y recursos, diagrama de red, camino crítico, revisar y aprobar la fecha de fin, completar el cronograma, consenso. **Roles**: *facilitador* (experimentado y de mente abierta; **no debería ser el LP**, porque tiene intereses en el resultado), *LP* (se concentra en el contenido y debe salir seguro del plan), *consultor* (experto externo), *tecnógrafo* (opera las herramientas y documenta), *equipo central*, *representante del cliente* (necesario, pero "problemático": puede presionar por plazos irreales o reabrir el alcance), *administradores de recursos* (dueños de la gente, expertos en estimar), *campeón del proyecto* (lo vende a la gerencia) y *administradores funcionales* (áreas que proveen o reciben). **Entregables**: WBS con estimaciones, diagrama de red, cronograma, asignación de recursos, documento y **Propuesta del Proyecto**.

7.2 WBS (Estructura de Descomposición del Trabajo)

WBS / EDT y actividad vs. tarea

Descripción **jerárquica** del trabajo para completar el proyecto, similar a una descomposición funcional: la **meta** se divide en **actividades** y las actividades en **tareas**. *Actividad*: unidad de nivel intermedio que se sigue descomponiendo ("Diseñar la aplicación"). *Tarea*: unidad del último nivel, que ya no se descompone y que se estima, asigna, secuencia y controla ("Diseñar la base de datos"). Es la convención de la cátedra; otros autores intercambian los términos.

Usos: diseñar y planificar el trabajo (todo el equipo ve cómo se define); mostrar su *arquitectura* (cómo se relacionan los ítems); estimar esfuerzo, tiempo y recursos en el último nivel y *agregarlos* hacia arriba; *informar el estado* ("Diseño al 60 %"). El WBS no es exclusivo del software (el apunte muestra uno para cubrir un puesto vacante) y suele numerarse jerárquicamente (1.0, 1.1, 1.2).

Construcción (responsabilidad del LP, a mano con stickers, pizarrón y paredes):

- **Top-down con todo el equipo**: desde el nivel 0 se particiona hasta que el trabajo está suficientemente definido; luego se secuencia y se buscan paralelismos. Estimaciones más exactas y todos ven todo, pero consume mucho tiempo.
- **Top-down con subequipos**: todos acuerdan el nivel 1 y cada subequipo descompone la actividad que mejor conoce (puede pedir ayuda). Más rápido, visión fragmentada.
- **Bottom-up**: tipo brainstorming; se listan tareas sueltas y se agrupan las relacionadas. Produce **granularidad despareja**; se mitiga con metodologías que dan listados de tareas típicas.

Criterios de nombre: por *sustantivos* (entregables: Definición, Análisis), por *verbos* (acciones: Definir alcance) u *organizacional* (áreas).

Cuándo parar de descomponer: los 6 criterios de completitud

1. **Estado medible**: en todo momento se sabe cuánto se hizo. "Codificar 10 componentes en 10 semanas": a la semana 4 no vale "40 %" ni "va bien"; vale "4 componentes codificados y aprobados".
2. **Acotada**: evento y fecha de comienzo, evento y fecha de fin.
3. **Entregable**: un signo visible de que se completó (producto, documento, autorización).
4. **Tiempo y costo estimables** fácilmente (y después se agregan).
5. **Duración aceptable**: en lo posible no más de **10 días** (excepciones como esperar una aprobación); una tarea larga se atrasa mucho antes de que alguien lo note.
6. **Independiente**: una vez iniciada sigue sin interrupciones ni inputs adicionales; si hay que esperar algo, conviene partirla.

7.3 Estimación: duración, esfuerzo, recursos y costos

Sobre cada tarea se estiman la **duración** (tiempo calendario), el **esfuerzo** (horas-persona o días-persona), los **recursos** (personas por perfil, facilidades, equipos, materiales) y los **costos**.

Agregados, dan la fecha de terminación, el presupuesto y el plan de recursos. En software se estima además el **tamaño** (LOC o PF), del que los modelos derivan el esfuerzo (§8.3).

Duración vs. esfuerzo

Tres variables influyen en la duración: la duración misma, el **esfuerzo** de trabajo y el **porcentaje de tiempo** diario que la persona le dedica:

$$\text{duración} = \frac{\text{esfuerzo}}{\text{porcentaje de dedicación}}$$

Las herramientas cargan dos y calculan la tercera. Ej.: 40 h-persona con una persona al 100 % (8 h/día) = 5 días; al 50 % = 10 días; en 4 días hacen falta $40/(8 \cdot 4) = 1{,}25$ personas. El **costo** depende del esfuerzo; la **fecha**, de la duración.

Crash point de una actividad

Su **duración mínima posible**: el punto desde el cual agregar recursos ya no la acorta (y puede alargarla). Cada persona nueva reduce cada vez menos por la coordinación, la comunicación y la capacitación. Al comprimir el cronograma se buscan tareas críticas que no llegaron a su crash point.

Técnicas de estimación de la duración.

- **Similitud con otras tareas**: lo que tardaron tareas parecidas, según la *memoria* de las personas (que suele recordar lo estimado más que lo real).
- **Datos históricos**: lo mismo, pero *registrado* en una base de datos.
- **Juicio experto**: consultores o vendedores; cuidado, al vendedor le conviene que todo parezca fácil.
- **Delphi**: cada miembro estima solo y se tabula (histograma de la primera pasada); los que cayeron en los **cuartiles exteriores justifican**; todos reestiman (segunda pasada) y se defienden de nuevo los extremos; una tercera pasada con ajustes finales; **el promedio de la tercera pasada es la estimación**. Aprovecha al grupo y evita que mande el jefe o que se pierda información que tenía uno solo.
- **Técnica de 3 puntos**: optimista O (todo sale bien), más probable M y pesimista P (falla lo que puede fallar): $E = (O + 4M + P)/6$, un promedio ponderado que le da 4 veces más peso al valor más probable.
- **Delphi de banda ancha** (*wideband*): Delphi donde cada uno da O, M y P; se **eliminan los extremos**, se **promedia** cada columna y se aplica la fórmula de 3 puntos.
- **Grupo de estimación** (De Marco): un coordinador reparte información, reúne al grupo, cada experto da una estimación *anónima* en rango, se circula el resumen y se repite hasta que el promedio es satisfactorio. Putnam: (cota inferior $+ 4 \cdot$ similares $+$ cota superior)/6.
- **Otros autores**: analogía; estimar el tamaño (LOC, PF de Albrecht) y aplicar un modelo algorítmico $a \cdot \text{KLOC}^{b} \cdot$ ajuste (COCOMO); estimación top-down o bottom-up. Y en ágil, **Planning Poker** (§2).

Delphi, 3 puntos y banda ancha con números

Delphi: primera pasada 10, 12, 11, 17, 19, 15 días: justifican los extremos (10 y 19); la estimación es el promedio de la tercera pasada. **3 puntos**: $O = 4$, $M = 6$, $P = 14 \Rightarrow E = (4 + 24 + 14)/6 = 7$ días (por encima de M por la cola pesimista). **Banda ancha** con cinco estimadores: se descartan los valores extremos de cada columna, quedan promedios $\bar{O} = 4$, $\bar{M} = 5{,}75$, $\bar{P} = 10{,}5$ y $E = (4 + 23 + 10{,}5)/6 = 6{,}25$ días.

Recursos y costos. Recursos: **humanos** (los más difíciles de estimar), **facilidades** (salas), **equipamiento**, **viáticos** y **materiales**. Las personas se planifican **por perfil** y luego se personalizan (vacaciones, gente compartida): herramienta, la **matriz por perfil** (§9.4). La **estimación de costos** predice tiempo, esfuerzo y perfiles para construir el sistema; se usa para planificar y para controlar. Proceso: con tiempo y recursos de cada tarea, costo unitario por cantidad, a nivel de tarea, y se agrega al proyecto; *materiales* por precio o por uso, y *laborales* por perfil con su valor horario. En software somos "notoriamente inexactos": cada proyecto difiere en dominio, hardware, herramientas y personal; somos más creadores que constructores; hay problemas *políticos* (la estimación se vuelve objetivo o se baja para ganar la licitación) y *técnicos* (no hay datos históricos).

7.4 Diagramas: Gantt, AON (PDM) y AOA (PERT)

Con las tareas, sus duraciones y el orden se construye un **diagrama de red**: una representación gráfica de la secuencia de las tareas (qué depende de qué y qué va en paralelo), de donde salen el comienzo más temprano de cada tarea y la fecha de fin.

- **Gantt** (Henry Gantt): diagrama de barras de doble entrada (tareas × tiempo), donde la longitud es la duración. Se dibujan primero las tareas sin predecesoras desde 0 y luego las que dependen de las ya dibujadas. *Ventajas*: muy sencillo, sin herramientas, cualquiera lo entiende (ideal para comunicar). *Desventajas*: no muestra bien dependencias, ni cómo terminar antes, ni si la asignación de recursos es efectiva; inmanejable con muchas tareas.
- **AON (Activity On the Node), método PDM**: cada tarea es un rectángulo (nodo) y las flechas son precedencias; se lee de izquierda a derecha, cada tarea (salvo inicio y fin) tiene una predecesora y una sucesora inmediatas, y la red es conexa. *Ventajas*: intuitivo, admite los **cuatro tipos de dependencias** y los lags, no necesita ficticias, estándar actual. *Desventajas*: sin escala de tiempo; maraña con muchas tareas.
- **AOA (Activity On the Arrow), método PERT**: cada **flecha** es una tarea y los nodos son **eventos** (inicio y fin). **Entre dos vértices no puede haber más de un vector**: se agregan **tareas ficticias** de duración 0. Cada vértice se valora con el **máximo** de (valor del vértice previo + duración) sobre las flechas que llegan; el camino crítico es el que da el valor del último vértice. *Ventajas*: destaca hitos, base probabilística. *Desventajas*: solo FC, ficticias confusas, difícil de modificar, **casi no se usa**.

Valoración de vértices en PERT (apunte)

A (12, sin pred.), B (2, pred. D), C (3, pred. B y F), D (8, pred. A), E (8, pred. D, B, F), F (13, pred. A). El vértice donde terminan A vale 12; donde terminan F y B recibe $12 + 13 = 25$ (por F) y $20 + 2 = 22$ (por D y B): se toma el mayor, **25**; el final recibe $25 + 8 = 33$ por E. Camino crítico **A**–**F**–**E**, 33 unidades.

7.5 Dependencias, restricciones y lags

Una **dependencia** es una relación entre una predecesora A y una sucesora B que restringe cuándo B puede comenzar o terminar. Se nombran "extremo de A – extremo de B":

Tipo	Regla	Ejemplo
FC / FI (fin a comienzo)	A debe estar completa antes de que B comience. La más común.	codificar el módulo A → testearlo
CC / II (comienzo a comienzo)	B comienza después de que A comenzó (juntas o con diferencia).	codificar el módulo A y generar sus datos de prueba
FF (fin a fin)	B no puede terminar antes de que termine A.	diseño de módulos y diseño de interfaces
CF / IF (comienzo a fin)	B no puede terminar antes de que A comience. La más rara.	discontinuar el sistema viejo (B) recién cuando arrancó el nuevo (A)

Las demoras entre tareas se modelan con **lags** ("registrar las encuestas" empieza 5 días después de

"enviarlas": CC con lag 5); un lag negativo es un **adelanto** (*lead*).

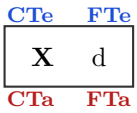
Las dependencias nacen de **cuatro tipos de restricciones** (cualquiera puede generar cualquier tipo de dependencia):

- **Técnicas**: la sucesora necesita un output de la predecesora (conviene FC). Subtipos: *discrecionales* (juicio del LP, por ejemplo para prevenir riesgos), *por mejores prácticas* (experiencia documentada; ej.: pasar una FC a CC para acortar), *lógicas* (organización razonable: la base de datos antes que las pantallas) y *por requerimiento único* (un recurso crítico compartido obliga a secuenciar con FC).
- **Gerenciales**: decisiones políticas de la empresa; a diferencia de las técnicas, **pueden revertirse**.
- **Inter-proyectos**: los entregables de un proyecto son necesarios para otro (proyectos grandes divididos en subproyectos).
- **De fechas**: fechas impuestas externamente; **deben evitarse**. De más leve a más grave: *no antes que*, *no después de*, *en esta fecha*.

7.6 Planificación temprana, tardía, ventana, márgenes y camino crítico

Se calculan dos planificaciones derivadas de las dependencias: la **temprana** (hacia adelante, *forward*: lo más temprano que cada tarea puede empezar y terminar) y la **tardía** (hacia atrás, *backward*: lo más tarde que puede empezar y terminar sin atrasar el proyecto). Combinándolas se obtienen la **ventana** de cada tarea y la **fecha de fin** del proyecto.

Convención de la cátedra: días discretos numerados desde 1. Una tarea de d días que empieza el día c ocupa los días $c, \ldots, c+d-1$, así que *termina* el día $c+d-1$, y su sucesora FC empieza *al día siguiente*. De ahí salen los ± 1. En el nodo, arriba (azul) va la temprana y abajo (rojo) la tardía.



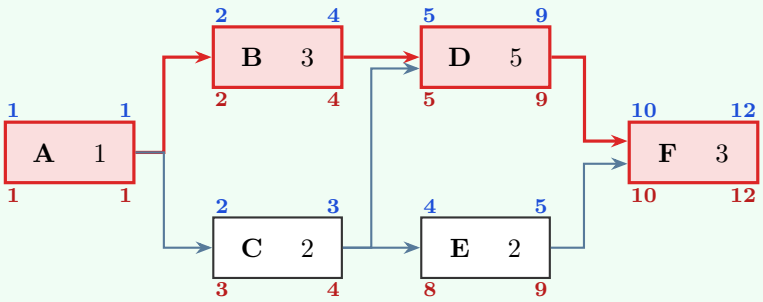
Concepto	Fórmula	Qué significa
Comienzo más temprano	$\text{CTe} = \text{máx}_{\text{pred}}\ \text{FTe} + 1$	cuando terminaron *todas* las predecesoras (manda la que termina más tarde); 1 si no tiene
Fin más temprano	$\text{FTe} = \text{CTe} + d - 1$	ej.: empieza el 45 y dura 20: termina el 64
Fin más tardío	$\text{FTa} = \text{mín}_{\text{suc}}\ \text{CTa} - 1$	sin atrasar a *ninguna* sucesora (manda la más urgente); si no tiene sucesoras, el **fin del proyecto**
Comienzo más tardío	$\text{CTa} = \text{FTa} - d + 1$	ej.: termina como tarde el 90 y dura 20: empieza el 71
Ventana	$V = \text{FTa} - \text{CTe} + 1$	período entre el comienzo más temprano y el fin más tardío
Margen total	$\text{MT} = V - d$	demora tolerable sin atrasar el **proyecto** (= FTa − FTe sin FF ni CF)
Margen libre	$\text{ML} = \text{mín}_{\text{suc}}\ \text{CTe} - \text{FTe} - 1$	demora tolerable sin atrasar el comienzo temprano de **ninguna sucesora**; si no tiene sucesoras, se usa fin del proyecto +1

El **margen** (*slack*) es la demora, en unidades de tiempo, que tolera una tarea sin atrasar el fin del proyecto. El **libre** es el margen "propio"; el **total** incluye el que comparte con sus sucesoras (si lo consume, ellas pierden el suyo). Siempre $\text{ML} \leq \text{MT}$.

Camino crítico

La secuencia de tareas que **determina la fecha de fin del proyecto**. Tres definiciones equivalentes: el **camino más largo** de la red; las tareas cuya planificación **temprana y tardía coinciden**; las tareas con **margen cero**. Cualquier demora en una tarea crítica demora el proyecto lo mismo. Puede haber **más de uno**. Se calcula enumerando caminos (redes chicas) o por márgenes.

Ejemplo completo del apunte



C: CTe $= 2$, FTa $= \min(\text{CTa}_D, \text{CTa}_E) - 1 = \min(5, 8) - 1 = 4$; $V = 4 - 2 + 1 = 3$, MT $= 3 - 2 = 1$, ML $= \min(5, 4) - 3 - 1 = 0$ (si C se atrasa un día, E ya no empieza el 4). **E**: $V = 9 - 4 + 1 = 6$, MT $=$ ML $= 4$. **Camino crítico** A–B–D–F, 12 días. (El apunte dice "ML de C $= 1$": es una errata; ese 1 es el MT.)

Dependencias CC, FF y CF en el cálculo. Con predecesora q y sucesora p:

Tipo	Hacia adelante	Hacia atrás	Holgura de q (para su ML)
FC	$\text{CTe}(p) \geq \text{FTe}(q) + 1$	$\text{FTa}(q) \leq \text{CTa}(p) - 1$	$\text{CTe}(p) - \text{FTe}(q) - 1$
CC	$\text{CTe}(p) \geq \text{CTe}(q)$ (sin $+1$)	$\text{CTa}(q) \leq \text{CTa}(p)$	$\text{CTe}(p) - \text{CTe}(q)$
FF	$\text{FTe}(p) \geq \text{FTe}(q)$	$\text{FTa}(q) \leq \text{FTa}(p)$	$\text{FTe}(p) - \text{FTe}(q)$
CF	$\text{FTe}(p) \geq \text{CTe}(q)$	$\text{CTa}(q) \leq \text{FTa}(p)$	$\text{FTe}(p) - \text{CTe}(q)$

Cómo resuelve la cátedra FF y CF (TP3, ejercicio 3)

Una FF o CF restringe el *fin* de la sucesora, no su comienzo. Entonces, si la tarea no tiene predecesoras FC o CC, **CTe** $= 1$ y **FTe** $= \max($**CTe** $+ d - 1$**, lo que impone la FF/CF)**. Ejemplo: I (2 días, H [FF], FTe(H) $= 34$): CTe $= 1$, FTe $= 34$; sin sucesoras, FTa $= 45$ (fin del proyecto); $V = 45$, **MT** $= V - d = 43$ (no FTa $-$ FTe $= 11$). K (9 días, J [CF], CTe(J) $= 25$): CTe $= 1$, FTe $= 25$, MT $= 36$. Cuidado también con el ML con sucesoras CC: si E tiene una sucesora F (CC) que empieza el mismo día, ML(E) $=$ CTe(F) $-$ CTe(E) $= 0$, aunque la fórmula FC daría un número negativo.

Compresión del cronograma. Si la fecha calculada es posterior a la requerida: buscar áreas para comprimir; pares de tareas en secuencia que se puedan hacer en paralelo (la sucesora empieza con un avance parcial de la predecesora: FC $\rightarrow$ CC con lag, **entregables parciales**); **actuar sobre el camino crítico** (acortar una tarea no crítica no acorta el proyecto); buscar tareas particionables. Consecuencias: más **riesgo** (retrabajo si la predecesora cambia), igual o más trabajo total, más coordinación, y puede **cambiar el camino crítico**. **Reserva gerencial**: tarea ficticia al final, del **5 al 10 %** de la duración; se gestiona para no usarla, motiva al equipo y sirve con contratos rígidos.

Propuesta de Proyecto y aprobación. Entregable final de la JPPS, para los gerentes senior: resumen ejecutivo (el POS expandido), antecedentes, objetivos, estrategia (y modelo de ciclo de vida), descripción detallada del trabajo (Gantt), resumen de tiempos y costos, apéndices. Suele

volver para revisión porque el costo/beneficio no cierra (eliminar funciones de poco beneficio), los riesgos son altos (otras soluciones), el costo excede el financiamiento (triángulo de alcance o fases) u otros proyectos compiten por los recursos.

En resumen JPPS con facilitador (no el LP) → WBS (actividades y tareas con los 6 criterios) → estimación (duración = esfuerzo/dedicación, crash point, Delphi, 3 puntos, banda ancha) → red AON con 4 tipos de dependencias → temprana, tardía, ventana, márgenes y camino crítico → compresión y reserva → Propuesta de Proyecto.

8 Métricas: Puntos de Función y COCOMO [TP4]

8.1 Por qué medir la especificación

La **especificación** describe *qué* debe hacer el sistema (no cómo): sus **funciones**, los **datos** que maneja, sus **interfaces** con usuarios y otros sistemas, y sus **restricciones** (no funcionales). Al medir los modelos de especificación se busca cuantificar, lo antes posible, el **tamaño funcional** (cuánta funcionalidad entrega), la **complejidad** y la **calidad de la especificación** (si es específica, completa y consistente), para estimar esfuerzo, costo y plazo, comparar proyectos y medir productividad. Las **líneas de código** (LOC, KLOC) son malas para eso: se conocen al final, dependen del lenguaje, penalizan a los lenguajes y programadores concisos y no tienen una definición única.

8.2 Puntos de Función de Albrecht

Puntos de Función de Albrecht

Miden el **tamaño funcional**: la funcionalidad que el sistema entrega **desde el punto de vista del usuario**, **independientemente del lenguaje y la tecnología**, y se pueden contar **desde la especificación**.

Se traza la **frontera** del sistema y se cuentan cinco tipos de componentes:

- **Entradas externas (EE)**: procesos en los que **se introducen datos** desde afuera y que normalmente *mantienen* un archivo interno (alta, modificación, baja: cada una cuenta).
- **Salidas externas (SE)**: procesos que **envían datos al exterior** con **información derivada** (cálculos, totales, promedios).
- **Consultas externas (CE)**: combinan una entrada y una salida (se pide y se devuelve) **sin afectar los almacenamientos** y **sin información derivada** (se muestran los datos tal cual).
- **Archivos internos (AI)**: **grupos lógicos de datos** relacionados, reconocibles por el usuario, que el sistema **mantiene** dentro de su frontera.
- **Archivos externos (AE)**: grupos de datos que el sistema **solo referencia**: los mantiene otro sistema.

Las dos confusiones típicas

SE vs. CE: si calcula algo, es salida ("ver el promedio del alumno"); si recupera tal cual, es consulta ("ver la ficha del alumno"). **AI vs. AE**: no importa dónde está la tabla sino *quién la mantiene*. Y "archivo" es un grupo lógico, no una tabla física.

Paso 2: complejidad de cada componente (baja, media o alta). En los ejercicios suele venir dada ("simple" = baja, "promedio" = media, "complejo" = alta). En un conteo real se usa la cantidad de **datos**, de **registros** (para archivos) y de **tipos de archivos** (para transacciones):

AI y AE			
Registros	1–19	20–50	51+
1	B	B	M
2–5	B	M	A
6+	M	A	A

SE y CE			
T. arch.	1–5	6–19	20+
0–1	B	B	M
2–3	B	M	A
4+	M	A	A

EE			
T. arch.	1–4	5–15	16+
0–1	B	B	M
2–3	B	M	A
4+	M	A	A

(Las columnas son la cantidad de datos.) Ej.: una EE con 12 datos que referencia 2 archivos es de complejidad media.

Paso 3: PFNA. Cada cantidad se multiplica por su peso y se suma todo:

Peso	Baja	Media	Alta
EE	3	4	6
SE	4	5	7
CE	3	4	6
AI	7	10	15
AE	5	7	10

Los **AI pesan más** (cada grupo mantenido implica altas, bajas, modificaciones e integridad); los AE pesan menos que los AI (solo se leen); las SE algo más que EE y CE (procesan); EE y CE pesan igual.

$$\text{PFNA} = \sum_{\text{tipo}} \sum_{\text{complejidad}} \text{cantidad} \times \text{peso}$$

Paso 4: ajuste. Dos sistemas con la misma funcionalidad pueden ser muy distintos de construir. Se califican **14 factores** con un grado de influencia de **0** (irrelevante) a **5** (esencial): (1) comunicación de datos, (2) procesamiento distribuido, (3) rendimiento, (4) configuración muy utilizada, (5) tasa de transacciones, (6) entrada de datos en línea, (7) eficiencia para el usuario final, (8) actualización en línea, (9) procesamiento complejo, (10) reusabilidad, (11) facilidad de instalación, (12) facilidad de operación, (13) múltiples sitios, (14) facilidad de cambio. Su suma es el **total de influencias** TI (0 a 70):

$$\boxed{\text{FCT} = 0{,}65 + 0{,}01 \times \text{TI}} \in [0{,}65;\ 1{,}35] \qquad \boxed{\text{PFA} = \text{PFNA} \times \text{FCT}}$$

TI = 35 es el punto neutro (FCT = 1); cada punto de TI por encima o por debajo de 35 mueve el resultado un 1 %.

Sistema S1 (resuelto por la cátedra en la presentación del TP4)

4 EE, 7 SE, 4 CE, 3 AI y 2 AE, todos de complejidad media: PFNA = 4·4+7·5+4·4+3·10+2·7 = 16 + 35 + 16 + 30 + 14 = **111**. Factores: irrelevante F1, F3, F5; 1 F2; 2 F12; promedio (3) F4, F6–F8, F10; 4 F9; esencial F11, F13, F14. Se despliegan (F6–F8 son tres factores) y se verifica que sean 14: TI = $0 \cdot 3 + 1 + 2 + 3 \cdot 5 + 4 + 5 \cdot 3 = 37$; FCT = 0,65 + 0,37 = 1,02; **PFA** = 111 × 1,02 = 113,22. (Del TP4: S2 da PFNA 83 y PFA 92,13; S3, 210 y 224,70.)

Errata de la presentación del TP4

Una diapositiva escribe "FCT = 0,65 ∗ 0,01∗ TI". Es una suma: 0,65 + 0,01· TI (con el producto daría 0,24, absurdo).

Usos: estimar esfuerzo con la productividad histórica (PF por persona-mes), alimentar COCOMO (convirtiendo a LOC), medir productividad y calidad comparables entre lenguajes, contratos. **Críticas**: subjetividad del conteo, pensados para sistemas de información (no tiempo real), el ajuste de ±35 % es discutido.

8.3 COCOMO 81

COCOMO (COnstructive COst MOdel, Boehm, 1981)

Modelo de estimación de costos. Estima el **esfuerzo** y la **duración** a partir del **tamaño**. Es "constructivo" porque sus ecuaciones son públicas.

$$E = a \cdot S^b \cdot F \ \text{(personas-mes)} \qquad D = c \cdot E^d \ \text{(meses)} \qquad P = E/D \ \text{(personas)}$$

S en **KDSI**: miles de instrucciones fuente *entregadas* (no cuentan comentarios, líneas en blanco ni código de prueba que el cliente no recibe).

$b > 1$ indica **deseconomía de escala**: duplicar el tamaño *más que duplica* el esfuerzo (más comunicación e integración). $d < 1$: la duración crece mucho más lento que el esfuerzo, porque se trabaja con más gente en paralelo.

Los tres modelos se usan según cuánto se sabe:

- **Básico**: se sabe muy poco; solo tamaño y modo; $F = 1$. Rápido y poco preciso.
- **Intermedio**: tras la especificación de requerimientos; F es el **producto de los multiplicadores de 15 conductores de costo** (EAF). Para proyectos "medios" (todo nominal) coincide con el básico.
- **Avanzado (detallado)**: con el diseño terminado y los módulos identificados; aplica el intermedio *por componente* y con multiplicadores *por fase*, y suma.

Los tres modos (el tipo de proyecto determina a, b, c, d):

Modo	**Características y ejemplos**	a	b	c	d
Orgánico	procesamiento de datos, uso de BD, transacciones y recuperación de datos; equipos chicos, entorno familiar, requisitos flexibles (facturación, gestión de alumnos, inventario)	2,4	1,05	2,5	0,38
Semi-embebido	intermedio: más procesamiento de transacciones, experiencia mixta, algunos requisitos rígidos (monitoreo de red, reservas, compilador, banca)	3,0	1,12	2,5	0,35
Embebido	software de tiempo real que es parte de un sistema mayor con hardware; restricciones rígidas que no se negocian (ascensores, aviónica, marcapasos, ABS)	3,6	1,20	2,5	0,32

De orgánico a embebido crecen a y b (a igual tamaño cuesta más y la deseconomía es mayor), c queda igual y d baja (proporcionalmente más gente en paralelo). Truco para elegir el modo: "si un requisito es difícil, ¿lo puedo negociar?": sí → orgánico; nunca → embebido; algunos → semi-embebido.

Errata del apunte

La diapositiva 68 cruza las filas de semi-embebido y embebido. El embebido es el más caro ($a = 3{,}6$, $b = 1{,}20$). En el examen, usá los coeficientes que te den.

Los 15 conductores de costo (escala Muy Baja, Baja, Nominal, Alta, Muy Alta, Extra Alta; nominal = 1,00): *del producto*: RELY (confiabilidad requerida: de una molestia a vidas humanas), DATA (tamaño de la BD respecto del programa), CPLX (complejidad); *del hardware*: TIME (uso del tiempo de CPU), STOR (de la memoria), VIRT (volatilidad de la máquina virtual), TURN (tiempo de respuesta para el programador); *del personal*: ACAP (capacidad de los analistas), AEXP (experiencia en la aplicación), PCAP (capacidad de los programadores como grupo), VEXP (experiencia en la máquina virtual), LEXP (en el lenguaje); *del proyecto*: MODP (prácticas modernas), TOOL (herramientas), SCED (restricción del cronograma: tanto apurar como estirar cuesta más). Subir producto o hardware *aumenta* el esfuerzo; subir personal, MODP o TOOL lo *reduce*. ACAP (de 1,46 a 0,71) es el de mayor rango: **las personas son el factor más influyente**. **Fases**: requerimientos (6–8 % del esfuerzo), diseño del producto (16–18 %), programación (48–68 %), prueba e integración (16–34 %). Pasos: estimar el tamaño y el esfuerzo inicial, revisar con los conductores, aplicar las herramientas del modelo. "No es sabio confiar ciegamente en el modelo, pero es menos sabio ignorarlo."

COCOMO básico: 32 KDSI en los tres modos

Orgánico: $E = 2{,}4 \cdot 32^{1{,}05} \approx 91{,}3$ PM; $D = 2{,}5 \cdot 91{,}3^{0{,}38} \approx 13{,}9$ meses; $P \approx 6{,}6$ personas. Semi-embebido: $E \approx 145{,}5$, $D \approx 14{,}3$, $P \approx 10{,}2$. Embebido: $E \approx 230{,}4$, $D \approx 14{,}3$, $P \approx 16{,}1$. La duración casi no cambia; el esfuerzo se multiplica por 2,5. Intermedio: con RELY alta 1,15, CPLX alta 1,15, ACAP alta 0,86 y TOOL baja 1,10 (el resto nominal), EAF $= 1{,}15 \cdot 1{,}15 \cdot 0{,}86 \cdot 1{,}10 \approx 1{,}25$: el esfuerzo nominal se multiplica por 1,25 (los coeficientes a y b del intermedio los da el enunciado).

8.4 COCOMO II y otros modelos

COCOMO 81 fue calibrado para cascada, lotes y lenguajes de los 70. **COCOMO II** (2000) tiene como **objetivos**: modelos acordes a las prácticas actuales; bases de datos y herramientas de costos para la mejora continua del modelo; y un marco analítico cuantitativo para evaluar el efecto de las mejoras tecnológicas en costos y planificaciones. **Qué lo diferencia**: sus modelos dependen de la *etapa* del ciclo de vida y cada uno usa otra medida de *tamaño*; incorpora reuso, OO, componentes y desarrollo iterativo; y el exponente b mide la economía de escala ($b < 1$ economía de escala, $b = 1$ balance, $b > 1$ deseconomía: más comunicación y problemas de integración).

Modelo	Cuándo	Tamaño
Application Composition	etapas tempranas, prototipos, aplicaciones con herramientas de alto nivel	**puntos objeto**
Early Design	tras los requerimientos, para evaluar arquitecturas alternativas	**puntos de función**
Post-Architecture	durante el desarrollo, con la arquitectura definida	**KSLOC**

Puntos objeto: se cuentan pantallas, reportes y componentes 3GL; se clasifican (simple, media, alta, según vistas o secciones y tablas de datos) y se pesan: pantalla 1/2/3, reporte 2/5/8, componente 3GL 10. La suma son los PO; con el reuso, NPO = PO × (100 − %reuso)/100; la productividad PROD vale 4, 7, 13, 25 o 50 NPO por persona-mes (de muy baja a muy alta experiencia y madurez de herramientas); y PM = NPO/PROD. Ej.: 5 pantallas simples, 3 medias, 4 reportes medios y 2 componentes 3GL dan PO = 51; con 20 % de reuso, NPO = 40,8; con PROD nominal 13, PM ≈ 3,1. **Early Design** convierte los PF a líneas con una tabla por lenguaje (Java 53, C 128, Cobol 91, planilla de cálculo 6: por eso las LOC miden mal la funcionalidad).

Otros modelos: SLIM (Putnam) y descendientes de COCOMO (COCOTS, COSYSMO…). **Problemas**: el tamaño es el factor determinante; la deseconomía de escala; Putnam: acortar la duración aumenta el esfuerzo; cada modelo funciona en el entorno del que salió; son sensibles al factor tecnológico; los datos de entrada son difíciles de obtener justo al principio. **Mejoras**: datos locales consistentes, **calibrar** el modelo con proyectos propios, un grupo de estimación independiente, reducir la subjetividad y **re-estimar** a medida que hay más información.

En resumen PF: EE, SE, CE, AI, AE por complejidad y peso → PFNA; 14 factores → TI → FCT = 0,65 + 0,01 TI → PFA. COCOMO 81: $E = aS^bF$, $D = cE^d$; básico ($F = 1$), intermedio (15 conductores), avanzado (por componente y fase); modos orgánico, semi-embebido y embebido. COCOMO II: puntos objeto, PF o KSLOC según la etapa, y b variable.

9 Lanzamiento: equipo, reglas, cambios y kick-off [TP5]

Con el plan aprobado todavía no se puede programar: falta decidir *quién* hace cada cosa, *cómo* trabaja junta esa gente y qué pasa cuando el cliente pida algo nuevo. El **lanzamiento** reúne cinco piezas: el **equipo**, las **reglas de trabajo**, el **proceso de gestión de cambios**, los **paquetes de trabajo** y la **finalización del cronograma**, presentadas en la reunión de **kick-off**.

9.1 Organización del equipo

Estructura organizativa y organizar

El fin de una estructura organizativa es **facilitar la cooperación para lograr un objetivo común**. Organizar es **identificar roles** y **asignar responsabilidades**. La organización adecuada **depende del tipo de proyecto** (un equipo de fútbol y uno de posta de natación se organizan distinto).

- **Según la comunicación**: menos comunicación → estructura **jerárquica** (ordenada, escala bien, lenta para problemas que cruzan áreas); más comunicación → estructura más **democrática** (más ideas y compromiso, más costo de coordinación). Los **grupos pequeños** tienen mayor cohesión y motivación y son buenos para problemas pequeños.
- **Tamaño**: COCOMO da el esfuerzo y de ahí la cantidad de personas (E/D); también influyen las características del software. Conviene **construir el grupo gradualmente**: pocos para requerimientos y diseño, más para la implementación, analizar e iterar.
- **Restricciones**: reorganizar es molesto (choca con la seguridad y la inercia de las personas); hace falta imaginación. El gerente equilibra cronograma y presupuesto, calidad del producto, motivación del personal y **crecimiento individual**. En proyectos largos importa la satisfacción personal para reducir la rotación.

Estructuras de ejemplo (los resultados se logran cuando cada uno sabe exactamente su responsabilidad):

- **Estructura I** (proyectos pequeños a medianos): gerente de proyecto → líder de proyecto → **hasta 5 programadores**. Cada uno tiene un trabajo específico y supervisión técnica. El **LP** da la función de control técnico y resuelve problemas de sistemas; su mayor responsabilidad es la **calidad del producto**. El **GP** da la conducción administrativa y maneja la comunicación con el exterior; su mayor responsabilidad es la **planificación y el control**.
- **Estructura II**: un LP no puede supervisar a más de 5 programadores; si hacen falta más, se intercalan **líderes de equipo**.
- **Estructura III**: el LP supervisa a algunos y un **programador senior** se hace cargo de un subgrupo (agranda el equipo sin otro nivel y hace crecer al senior).

	Cómo funciona	**Ventajas**	**Desventajas**
Funcional	los equipos se arman por área: un grupo para cada tipo de software	es fácil ubicar al responsable de un problema	difícil coordinar recursos de otros grupos; trabajar siempre con el mismo equipo se vuelve rutinario
Matricial	las personas pertenecen a grupos permanentes con un **gerente de línea (GL)**; el LP elige a los mejores, los negocia con el GL, los tiene full time durante el proyecto y luego vuelven; el GP le paga al GL	los especialistas se comparten entre proyectos y conservan su "hogar" técnico	cada persona tiene **dos jefes**; funciona si GP y GL tienen igual autoridad, pero el GL es el permanente y su opinión pesa más

Reclutamiento: las tres componentes del personal:

- **Equipo central**: está desde el comienzo y participa de la JPPS; no siempre a tiempo completo. Cuesta conseguirlo (portafolios agresivos, gente sobrecargada, alta demanda laboral). Se busca: compromiso con el trabajo, responsabilidad compartida, flexibilidad, orientación a tareas con restricciones, confianza, mente abierta, habilidad para trabajar entre estructuras y autoridades, y manejo de herramientas.

- **Equipo del cliente**: **no se elige** (hay que ser cauteloso), participa desde el kick-off, con dedicación parcial.
- **Equipo contratado**: externos, por falta de personal o capacitación; el **tiempo del contrato no varía**, deben empaparse del proyecto, el compromiso y la calidad pueden ser un problema y requieren **más supervisión**.

El **balance** del equipo es un factor crítico. El inventario de **Kolb** clasifica a las personas en *asimiladoras* (recolectan y representan datos en forma lógica), *divergentes* (ven alternativas y perspectivas), *acomodadoras* (orientadas a resultados, prácticas) y *convergentes* (resuelven problemas, encuentran soluciones pero no las implementan). Un buen equipo tiene de los cuatro.

9.2 Reglas de trabajo en grupo

Definen cómo trabaja el grupo y se acuerdan *antes* de los problemas. Cubren seis situaciones:

- **Solución de problemas** (Couger, 5 pasos): delinear la oportunidad y definir el problema; compilar información; generar ideas; evaluarlas y priorizarlas; desarrollar un plan.
- **Toma de decisiones**: **directivo** (decide la autoridad: expeditivo, pero puede faltarle información y genera resistencia), **participativo** (contribuyen todos: confianza y compromiso, relega responsabilidad del LP, lento) y **consultativo** (decide la autoridad después de consultar: "políticamente una buena estrategia"). Modelo de Morris y Sashkin (6 fases): definir la situación, generar alternativas, ideas para actuar (métricas), plan de acción, evaluar la decisión y evaluar el resultado.
- **Resolución de conflictos** (opiniones distintas, o el cliente decide algo que el equipo no comparte): **evitar** o no hacer nada (no tolerable), **combatir** (a veces útil, sube el estrés), **colaborar** (buscar *win–win*: la constructiva).
- **Elaboración de consenso**: el equipo llega a un punto en que nadie tiene desacuerdos serios; **no es votar**; se usa cuando la decisión no es clara y no hay serias divergencias.
- **Brainstorming**: para cuando faltan ideas; todos presentan ideas sin discusión, después se combinan y revisan; no apurar el proceso. El equipo debe saber cómo lo conduce el LP y qué se hace con el resultado.
- **Reuniones de equipo**: *diarias de estado* (cortas), *de resolución de problemas* (solo los involucrados) y *de revisión del proyecto* (formales, en hitos); para cada una, frecuencia, agenda, coordinador y minutas.

9.3 Administración de cambios de alcance

Sin importar el ciclo de vida habrá pedidos de cambio, del cliente o del equipo. Si se aceptan de palabra, el alcance crece sin ajustar plazos ni costos (*scope creep*). Por eso se acuerda un proceso en el lanzamiento.

Cómo se maneja un pedido que modifica el alcance

1. **Pedido de cambio**: *todo* cambio se documenta (memo o formulario) y abre una **ronda de COS** hasta entenderlo.
2. **Revisión del pedido**: se rechaza, se devuelve para revisar o se acepta para análisis.
3. **Análisis del impacto**: el equipo identifica **cursos de acción alternativos** y su efecto en cronograma, recursos y entregables.
4. **Revisión del impacto**: **el cliente elige** la alternativa (o rechaza, o pide revisar).
5. Se aprueba, se implementa y se actualiza el plan (la línea base).

Seis resultados posibles del análisis: (1) se incorpora con los recursos y el cronograma actuales; (2) requiere **extender el cronograma**; (3) requiere **recursos adicionales**; (4) requiere recursos y extensión; (5) requiere una **revisión completa de los entregables** (repriorizar: entra uno, sale otro); (6) **no puede acomodarse** sin un cambio significativo del proyecto. El equipo no decide si

el cambio entra ni lo absorbe en silencio: muestra su precio y decide quien controla el alcance.

9.4 Paquetes de trabajo, cronograma final y matriz por perfil

Paquete de trabajo: descripción, hecha por el **responsable de cada tarea**, de cómo planea completarla en el tiempo establecido. Es **un seguro para el LP**: si esa persona no está, otro sigue el plan escrito. Puede ser un listado de acciones o un mini-proyecto con su red (asignación de paquete de trabajo o descripción del grupo de trabajo); hay que balancear detalle y papeleo. **Finalizar el cronograma**: pasar de perfiles a personas concretas, con su disponibilidad real, nivelando las sobreasignaciones.

Matriz por perfil y recurso crítico

(1) **Persona** × **Perfil**: qué perfiles cubre cada persona; contando por columna se ve cuántas personas cubren cada perfil. (2) **Actividad** × **Perfil**: qué perfil necesita cada actividad (sale de la planificación). (3) Combinadas, **Persona** × **Actividad**: para cada actividad, los **candidatos** son quienes tienen el perfil; se elige uno (en el ejemplo de la cátedra, el más barato). Un **recurso crítico** es el **único** que cubre un perfil necesario: si se enferma, renuncia o lo reasignan, sus actividades se frenan. No significa "el más caro" ni "el más experto".

TP5, ejercicio 2 (resuelto por la cátedra)

Persona	Doc	Redes	IT	Procesos	Software	Testeo	$/sem
Rosario Bléfari	✓						8.000
Luis Hamilton	✓						9.000
Carlos Leclerc		✓					14.000
Tigre Ulli		✓					14.000
Carlos Busqued			✓				11.200
Lali Espósito				✓			12.800
María Epumer / Paula Maffia					✓		12.800
Nina Suarez						✓	10.000
Dillom						✓	14.000
Personas por perfil	2	2	**1**	**1**	2	2	

Críticos: Busqued (único IT) y Espósito (única de procesos). Con seis actividades (una por perfil) y eligiendo al más barato de cada perfil, el equipo cuesta $68.800 por semana. Matiz: si una actividad necesitara dos personas del mismo perfil a la vez, ese perfil también sería crítico. Qué hacer: paquetes de trabajo bien escritos, capacitar a otro, no asignarle dos tareas simultáneas, tener un contratado identificado.

Asignar personas a tareas (TP5, ejercicio 1): armar las tareas, asignar por perfil, estimar horas por persona y por tarea, y verificar la carga (nadie por encima de su disponibilidad semanal); el LP supervisa como máximo a 5 personas, conviene dar a quien aspira a LP tareas que lo hagan crecer, y el equipo del cliente participa con dedicación parcial en lo que solo él sabe (relevamiento, validación, capacitación).

9.5 La reunión de kick-off

Kick-off (reunión de lanzamiento)

Es el **anuncio formal de que el proyecto fue planeado y aprobado para su ejecución**. Se hace **después de planificar y antes de cualquier trabajo efectivo**. Tiene dos partes: la **dirigida por el sponsor** (breve, ante gerentes senior e interesados, donde se presenta el POS: es una exhibición para la organización) y la **dirigida por el LP** (resto del día: sesión de trabajo del equipo).

Propósitos: presentar al sponsor; presentar la importancia del proyecto (el cliente); presentar

el proyecto (cliente y LP); presentar a los miembros del equipo; realizar el **PDS** (enunciado de definición del proyecto; las diapositivas solo lo nombran); establecer las reglas de trabajo; revisar el plan; escribir los paquetes de trabajo. **Por qué dedicarle tiempo**: no es solo para conocerse; da **legitimidad** (el sponsor respalda en público), **alinea** a todos, fija las reglas antes de los conflictos, convierte el plan del LP en el plan **del equipo** (compromiso) y marca el punto de inicio desde el cual se mide el avance.

> **En resumen** La organización depende del proyecto: jerárquica o democrática; estructuras I, II y III (un LP, hasta 5 programadores); funcional o matricial (dos jefes). Personal central, del cliente y contratado; balance de Kolb. Seis reglas de grupo. Los cambios se piden por escrito, se analizan con alternativas y decide el cliente (seis resultados). Paquetes de trabajo como seguro; matriz por perfil para asignar y detectar recursos críticos. Kick-off en dos partes.

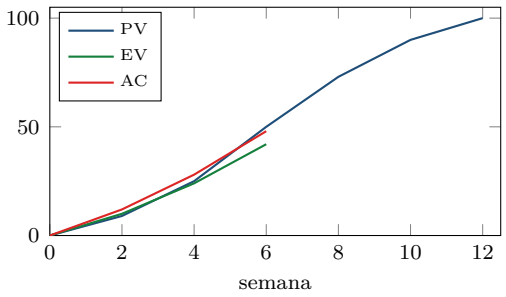
10 Monitoreo, control y cierre [TP6 partes I–II]

10.1 Monitoreo y control

El monitoreo y control asegura que el proyecto se mantenga dentro del **cronograma** y el **presupuesto** haciendo las **tareas previstas**: *monitorear* es medir y comparar con el plan; *controlar* es actuar ante los desvíos (acciones correctivas, replanificación, pedidos de cambio). Herramientas: programa de métricas, reportes, bitácora de pendientes, gestión de stakeholders y reuniones de avance.

- **Programa de métricas**: establece el *porqué* (primero: qué hay que controlar según los objetivos), el *qué* (qué métricas), el *dónde y cuándo* (una vez o repetidas) y el *quién y cómo* (herramientas y personas).
- **Sistema de reportes**: información completa, precisa y a tiempo, sin demasiada sobrecarga, de rápida aceptación, que advierta los problemas pendientes y fácil de entender. Tipos: **periódico** (un período: planeado vs. realizado, razones y medidas), **acumulativo** (toda la historia, tendencias), **de excepción** (solo las variaciones, para gerentes), **semáforo** (verde: todo según el plan, se puede no leer; amarillo: hay un problema descrito en el reporte; rojo: fuera de control sin plan correctivo, hay que analizarlo) y **de variantes** (diferencias planeado vs. real en tabla o gráfico). El **Gantt de avance** marca dentro de cada barra la proporción realizada.
- **Carta de tendencia de hitos**: grafica, para cada hito, cuántos días antes o después de lo planeado se cumplió (o se estima cumplir). Patrones: **retrasos sucesivos** (cada hito más tarde que el anterior; 4 o más piden acción correctiva), **cambios radicales** (un salto grande: renegociar el cronograma) y **corridas sucesivas** (7 o más del mismo lado: cambio permanente, tendencia).

Gestión del valor obtenido (EVM): mide la performance de forma objetiva **combinando alcance, cronograma y presupuesto** en una misma unidad, lo que permite pronosticar. Sin el EV, comparar lo planeado con lo gastado no dice nada ("gastamos menos" puede ser "hicimos menos"). **PV**: valor planificado del trabajo a cada fecha (su total es el BAC). **EV**: valor, a precio de presupuesto, del trabajo hecho (p. ej. la suma de los PV de las tareas concluidas). **AC**: costo real.



$$SV = EV - PV \qquad CV = EV - AC \qquad SPI = EV/PV \qquad CPI = EV/AC$$

$SPI < 1$: **atraso**; $CPI < 1$: lo hecho **cuesta más** de lo que vale. Ejemplo del gráfico (semana 6): PV = 50, EV = 42, AC = 48; SPI = 0,84 (va atrasado), CPI = 0,875 (sobrecosto). Comparando

solo PV y AC parecía ir bien. Proyectando PV sobre EV se estima la fecha de fin; proyectando AC, el costo final.

- **Bitácora de pendientes** (*issues log*): documento dinámico con todos los problemas *ya ocurridos* y no resueltos: problema, solución propuesta, impacto si no se soluciona, estado. Un riesgo materializado pasa de la tabla de riesgos a la bitácora.
- **Gestión de stakeholders**: identificar a quienes tienen interés o influencia en el proyecto y comunicarse efectivamente con ellos. *Objetivos*: identificarlos, definir sus intereses e influencias, comprometerlos y que se **apropien** del proyecto. *Factores críticos*: participación, compromiso, posesión y apoyo. *Beneficios*: satisfacción del cliente, entendimiento de quién gana o pierde, menos conflictos. *Etapas*: **identificación** (clientes, patrocinadores, afectados, proveedores, reguladores, equipos), **análisis** (intereses e impacto; **matriz de impacto** vs. interés: fuerte compromiso, consulta activa, mantener el interés, mantener informado), **planificación** (plan de compromiso) y **comunicación y gestión** (de doble vía, con un **plan de comunicación**: qué, cómo, quién y cuándo, y canales como talleres, boletines, informes).
- **Reuniones de avance**: semanales (además de dailies de 15 minutos en ágil). Orden: el gerente o cliente informa cambios; el LP informa la salud del proyecto; los responsables de paquetes reportan; se revisan los problemas abiertos; se identifican problemas nuevos y se asignan; se cierra y se anuncia la próxima.

10.2 Cierre del proyecto

En el cierre se evalúa lo realizado y se documenta la experiencia. Seis actividades:

1. **Aceptación de los entregables**: **el cliente decide** cuándo está terminado; el LP **demuestra** que se cumplió la especificación (informal en proyectos chicos, ceremonial en grandes; incluye el test de aceptación si se especificó).
2. **Instalación**: *en fases* (por grupos de entregables); *en un corte* o *big bang* (se reemplaza todo de una vez, tras probar en un ambiente similar; el más riesgoso); *en paralelo* (nuevo y viejo conviven y se comparan; el más seguro y caro); *por unidad de negocio* (una por vez; útil con sucursales distantes).
3. **Documentación**: referencia para cambios futuros, registro histórico para estimar, entrenamiento de nuevos LP y evaluación del equipo.
4. **Auditoría post-implementación**: evalúa las metas y logros contra lo planeado y presupuestado, la satisfacción del cliente y las lecciones aprendidas. Casi no se hace porque **no es facturable**, pero la organización aprende mucho.
5. **Reporte final**: la historia de la ejecución (éxito, organización, técnicas, fortalezas y debilidades).
6. **Celebración del éxito**: reconocimiento al equipo.

En resumen Monitorear es medir y comparar; controlar es actuar. Programa de métricas (porqué primero), cinco tipos de reportes, carta de hitos (4 retrasos sucesivos, cambio radical, 7 corridas), valor ganado (SPI y CPI menores que 1 son malos), bitácora de pendientes, stakeholders en cuatro etapas y reuniones de avance. Cierre: aceptación, instalación, documentación, auditoría post-implementación, reporte final y celebración.

11 Calidad: Fenton, ISO, CMM y auditoría [TP6 III–VI]

11.1 Atributos externos y definiciones de calidad

Ojo: esta subsección y la siguiente no tienen diapositivas. Se incluyen porque el TP6 las pregunta, y salen del libro de Fenton que cita el enunciado (igual que la auditoría, que sale de Weber). Fenton mide tres clases de entidades: **procesos**, **productos** y **recursos**. Un atributo **interno** se mide solo a partir de la entidad (tamaño, complejidad ciclomática, acoplamiento, defectos hallados

en una inspección); uno **externo**, en relación con su **entorno**, observando su comportamiento (confiabilidad, usabilidad, mantenibilidad, eficiencia, portabilidad; la productividad de una persona). **Utilidad de medir los externos**: son los que le importan a usuarios y gerentes; permiten fijar y verificar requisitos de calidad objetivos ("tiempo medio entre fallas mayor que 500 h" en vez de "que sea confiable"), comparar productos y proveedores, evaluar el efecto de cambios de proceso y validar las métricas internas. **Dificultad**: se miden tarde (cuando el producto existe y se usa); por eso los internos se usan como **predictores**.

Definiciones (visiones) de la calidad que presenta Fenton: *trascendental* (se reconoce pero no se define), *del usuario* (adecuación al propósito), *de manufactura* (conformidad con la especificación), *del producto* (características internas) y *basada en el valor* (lo que el cliente está dispuesto a pagar). En la práctica la industria adoptó una visión **basada en defectos**. **Enfoques para monitorear la calidad**: un **modelo fijo** (adoptar un modelo publicado, como McCall o ISO 9126, y medir lo que dice) o un **modelo propio** (acordar con el cliente qué atributos importan y cómo medirlos; Gilb: todo atributo con escala, nivel planeado y mínimo aceptable).

Modelo de McCall (1977): factor — criterio — métrica

Jerárquico: los **factores** (externos, de alto nivel) se descomponen en **criterios** (internos) que se miden con **métricas**. Once factores en tres perspectivas:

- **Operación del producto**: *corrección* (¿hace lo que quiero?: trazabilidad, completitud, consistencia), *confiabilidad* (¿siempre y con precisión?: tolerancia a errores, consistencia, exactitud, simplicidad), *eficiencia* (de ejecución y de almacenamiento), *integridad* (¿es seguro?: control y auditoría de acceso), *usabilidad* (operabilidad, entrenamiento, comunicatividad).
- **Revisión del producto**: *mantenibilidad* (¿puedo corregirlo?: consistencia, simplicidad, concisión, autodescripción, modularidad), *flexibilidad* (¿puedo cambiarlo?: autodescripción, expansibilidad, generalidad, modularidad), *facilidad de prueba* (simplicidad, instrumentación, autodescripción, modularidad).
- **Transición del producto**: *portabilidad* (¿en otra máquina?: autodescripción, modularidad, independencia del software y de la máquina), *reusabilidad* (las mismas más generalidad), *interoperabilidad* (modularidad, comunalidad de comunicaciones y de datos).

Cómo se analiza cada criterio: se le definen métricas (sobre todo **listas de verificación** puntuadas por un revisor, p. ej. de 0 a 10, y algunas medidas objetivas), cada factor se calcula como una **combinación ponderada** $F = c_1 m_1 + \cdots + c_n m_n$ con pesos obtenidos por **regresión** con datos históricos, y se **valida** que la métrica prediga el factor; el análisis es específico de la organización. **Un único modelo**: *ventajas*: vocabulario común, comparar productos y proveedores, checklist que no olvida dimensiones, se aplica directo, base de certificación; *desventajas*: ningún modelo sirve igual para todos los dominios, pesos arbitrarios o no validados, mezcla atributos internos y externos, falsa objetividad, factores vagos.

11.2 Defectos, densidad, spoilage, usabilidad, mantenimiento y productividad

Error (equivocación humana) → **falta** (*fault*: la marca en el artefacto) → **falla** (*failure*: comportamiento incorrecto observado al ejecutar). Una falta puede no producir nunca una falla. **Bug**: término coloquial para una falta en el *código*. **Defecto**: más amplio, cualquier anomalía en *cualquier artefacto* (requisitos, diseño, código, documentación), se haya manifestado o no. Todo bug es un defecto; no todo defecto es un bug.

$$\text{Densidad de defectos} = \frac{\text{defectos conocidos}}{\text{tamaño (KLOC)}} \qquad \text{System spoilage} = \frac{\text{t. de corrección post-entrega}}{\text{t. total de desarrollo}}$$

La **densidad** es la métrica de calidad más usada. Al *desarrollador* le sirve para comparar módulos y ubicar los problemáticos, seguir versiones, decidir si se puede entregar y evaluar mejoras de proceso. Al *usuario* le da una idea general, pero a él le importan las *fallas en su uso*: pocas faltas

causan casi todas las fallas, así que la frecuencia de fallas (tiempo medio entre fallas) le dice más. **Consideraciones**: qué cuenta como defecto (severidades, pre o post entrega), defectos conocidos $\neq$ existentes (fijar una ventana de tiempo de uso), cómo se cuentan las LOC (o usar PF), comparar solo con las mismas reglas, y el efecto sobre las personas si se usa para evaluarlas. El **spoilage** mide la calidad por el *costo* de los defectos: qué parte del esfuerzo se "echa a perder" corrigiendo.

- **Usabilidad**: se mide con usuarios reales y tareas representativas: *efectividad* (% de tareas completadas, errores), *eficiencia* (tiempo por tarea, tiempo de aprendizaje, consultas a la ayuda) y *satisfacción* (cuestionarios). Los sustitutos internos (cantidad de pantallas de ayuda) no probaron predecirla.
- **Tipos de mantenimiento**: **correctivo** (corregir defectos), **adaptativo** (adaptar a cambios del entorno: SO, hardware, leyes), **perfectivo** (mejoras o funciones pedidas, rendimiento) y **preventivo** (mejorar la estructura para el futuro: refactorizar). La mayor parte del esfuerzo es perfectivo y adaptativo. **Mantenibilidad**: atributos *externos* (el proceso de mantenimiento: **tiempo medio de reparación**, descompuesto en reconocer el problema, demora administrativa, reunir herramientas, analizar, especificar el cambio y hacerlo) e *internos* como predictores (complejidad ciclomática, tamaño, acoplamiento y cohesión, **legibilidad**).
- **Índice Fog** (Gunning), legibilidad de documentos: Fog $= 0{,}4\,(\text{palabras/oraciones} + 100 \cdot$ palabras de 3 o más sílabas/palabras): años de educación para entender el texto (12 secundario, 13–16 universitario). En castellano infla los valores (palabras más largas); sirve para comparar versiones. Se baja con oraciones cortas y palabras cortas.
- **Productividad**: producción sobre esfuerzo (LOC o PF por persona-mes); atributo externo de un recurso. Problemas: qué se cuenta como producción (las LOC premian el código verboso, los PF no captan la dificultad técnica), la calidad y el reuso no entran en el cociente, qué esfuerzo se cuenta, gran variabilidad entre personas y proyectos, y la gente optimiza la métrica si se la evalúa con ella.
- **Estructura del equipo**: influye por la **comunicación**: la producción crece linealmente con las personas, los canales con el cuadrado. Fenton modela el equipo como un **grafo** (nodos = personas, aristas = canales): *tamaño* n; *nivel de comunicación del equipo* e (aristas); *densidad de comunicaciones* $e/[n(n-1)/2]$; *nivel de comunicación individual* = grado de cada nodo; *nivel promedio* $= 2e/n$. Ej.: LP conectado con A, B, C, D, más A–B y C–D: $n = 5$, $e = 6$, densidad 0,6, grados LP 4 y el resto 2, promedio 2,4 (el LP es un cuello de botella).

11.3 Normas ISO

Calidad según ISO

La **totalidad de rasgos y características de un producto o servicio que se basan en su habilidad de satisfacer necesidades enunciadas o implícitas**. Las *enunciadas* están en el contrato; las *implícitas*, no, pero cuentan. TQM: la calidad la define la **percepción del cliente**, que cambia constantemente.

ISO busca estructurar las operaciones para potenciar al personal, reducir burocracia y satisfacer clientes, y dar **interfaces comunes** entre organizaciones con un aseguramiento independiente (no cada cliente audita a cada proveedor). La **administración de calidad** (ISO 9000) trata de cómo administrar la organización para satisfacer al cliente: mejora continua, aprender de errores, procesos proactivos, procesos visibles y cambios controlados. Familia (versión 1994 del apunte): **ISO 9001** (diseño, desarrollo, producción, instalación y servicio; **con la que se certifica**), 9002 (producción, instalación y servicio), 9003 (inspección y pruebas finales), **9000-3** (guía para aplicar 9001 al software), 9004-4 (mejora), 9004-2 (servicios), 10011 (auditar sistemas de calidad), 10013 (manuales de calidad). El **QMS** define cómo se supervisa el trabajo diario con procesos predefinidos (pirámide: sistema de calidad, metodología y planificación, procedimientos y estándares). ISO 9001 agrupa sus requisitos en tres colecciones:

- **Sistema de Administración de Calidad**: *responsabilidad de la gerencia* (política de calidad firmada, estructura con roles, revisión por la dirección; sistema de calidad documentado en manuales, procedimientos y **planes de calidad**); *auditoría interna de calidad* (revisión periódica y sistemática, acciones correctivas, buscar mejoras); *registros de control de calidad* (la evidencia, archivada y consultable); *control de datos y documentos* (autorizados, con distribución y control de cambios); *entrenamiento* (necesidades y registros); *acciones correctivas y preventivas*.
- **Identificación y trazabilidad del producto**: control de los productos *provistos por el cliente*, de los *no conformes*, de los *desarrollados in-house*, de los *comprados y subcontratados*, y *almacenaje y entrega*. En software se traduce en gestión de configuración.
- **Administración del desarrollo** (IEEE 1298 + ISO 9001): objetivos del *ciclo de vida*, *inspección y testeo* (con mediciones, equipamiento y estado de las pruebas) y *administración del proyecto*, bajo un control de proceso que abarca revisión de contratos, planificación, diseño (especificación y validación) e instalación. No impone un ciclo de vida.

Calidad en ISO 9126 (producto de software)

La **totalidad de rasgos y características de un producto de software que influyen en su capacidad de satisfacer necesidades enunciadas o implícitas**, descompuesta en **seis características** con sus subcaracterísticas (cada una incluye "cumplimiento" de normas):

- **Funcionalidad**: adecuación, exactitud, interoperabilidad, seguridad de acceso.
- **Fiabilidad**: madurez, tolerancia a fallos, capacidad de recuperación.
- **Usabilidad**: capacidad de ser entendido, aprendido y operado; atracción.
- **Eficiencia**: comportamiento temporal, utilización de recursos.
- **Mantenibilidad**: capacidad de ser analizado y cambiado, estabilidad, capacidad de ser testeado.
- **Portabilidad**: adaptabilidad, instalabilidad, coexistencia, capacidad de reemplazar.

Permite evaluar y comparar productos sobre la misma base. Distingue calidad **interna** (propiedades estáticas, inspección), **externa** (dinámicas, ejecutando) y **en uso** (satisface al usuario en su entorno). No da métricas: la **ISO 14598** define el *proceso de evaluación*. La serie **ISO 25000 (SQuaRE)** las reemplaza, con cinco divisiones: gestión (2500n), modelo de calidad (2501n), medición (2502n), requisitos (2503n) y evaluación (2504n).

11.4 CMM y CMMI

CMM (Capability Maturity Model, SEI de Carnegie Mellon) describe los elementos de un proceso de software efectivo y un **camino evolutivo** de un proceso *ad hoc* a uno maduro. A diferencia de ISO 9001 (que dice *si* se cumple), CMM dice *qué tan madura* es la organización y qué mejorar después. **Estructura**: los *niveles de madurez* indican la *capacidad del proceso* (el rango de resultados esperables, que permite predecir el próximo proyecto); cada nivel (salvo el 1) contiene *áreas de proceso clave* (APC, cada una en un solo nivel), que logran *objetivos*; se organizan en *características comunes* (compromiso para realizar, habilidad para realizar, actividades realizadas, medición y análisis, verificación de la implementación; las cuatro que no son actividades dan la **institucionalización**), que contienen *prácticas clave*.

N	Nivel	Qué lo caracteriza	APC del nivel
1	Inicial	proceso ad hoc; en crisis se abandona lo planificado; el éxito depende de individuos héroes; capacidad **impredecible**	—
2	Repetible	políticas y procedimientos de gestión de proyectos; se planifica con la experiencia de proyectos similares para **repetir éxitos**; se controlan costos, cronogramas y funcionalidad; capacidad **disciplinada**	administración de requerimientos, planificación y seguimiento del proyecto, subcontratos, aseguramiento de calidad, configuración
3	Definido	**proceso estándar de la organización** documentado (ingeniería y gestión), un grupo responsable del proceso, entrenamiento; cada proyecto lo adapta; capacidad **estándar y consistente**	foco y definición del proceso de la organización, entrenamiento, gestión integrada, ingeniería de producto, coordinación entre grupos, revisiones por pares
4	Gestionado	objetivos de calidad **cuantitativos**; se mide productividad y calidad con una base de datos de procesos; capacidad **predecible**	gestión cuantitativa del proceso, gestión de la calidad del software
5	Optimizado	**mejora continua**; se identifican debilidades para **prevenir defectos**; se analizan causas e innovaciones	prevención de defectos, cambios de tecnología, cambios de proceso

Cómo se evoluciona: **no se saltean niveles** (cada uno es la base del siguiente); hay que implementar *todas* las APC del nivel siguiente y satisfacer sus objetivos, **institucionalizándolas** (compromiso, habilidad, medición, verificación), no solo haciendo las actividades. De 1 a 2 se impone disciplina por proyecto; de 2 a 3 se estandariza en la organización; de 3 a 4 se mide; de 4 a 5 se previene y mejora.

CMMI (Capability Maturity Model Integration): había varios modelos (CMM-SW, de personas, de adquisición, de ingeniería de sistemas, de desarrollo integrado...) con estructuras distintas; el SEI publicó CMMI para **integrar** CMM-SW, SE-CMM e IPD-CMM. Se basa en dos conceptos: **madurez**, atributo de la *organización* (niveles 1 a 5: inicial, gestionado, definido, cuantitativamente gestionado, optimizado), y **capacidad**, atributo de cada *proceso* (niveles 0 a 5: incompleto, ejecutado, gestionado, definido, cuantitativamente gestionado, optimizado).

11.5 Auditoría de sistemas de información

Auditoría de sistemas de información (Weber)

Proceso de **recolectar y evaluar evidencia** para determinar si un sistema informático **salvaguarda los activos**, **mantiene la integridad de los datos**, **logra eficazmente** los objetivos de la organización y **usa eficientemente** los recursos.

Objetivos de quien se somete a auditorías: esos cuatro, más cumplimiento legal y regulatorio, confianza de terceros (clientes, accionistas), detectar y disuadir abusos y obtener recomendaciones. Se justifican por el costo de perder datos, de decidir con información incorrecta, del abuso, de los errores de cómputo, el valor de los activos y la privacidad.

Abuso computacional: todo incidente asociado a la tecnología en el que una víctima sufre (o pudo sufrir) una pérdida y el perpetrador obtiene (o pudo obtener) un beneficio **intencionalmente**.

Tipo	Qué es	Ejemplos
Hacking	acceso lógico no autorizado para leer, modificar o destruir	robar la base de clientes por una vulnerabilidad; un ex empleado entra con una contraseña no dada de baja
Virus y código malicioso	programas que se propagan y dañan (gusanos, troyanos, bombas lógicas, ransomware)	ransomware que cifra un hospital; bomba lógica que borra la nómina
Acceso físico ilegal	entrar a instalaciones o llevarse equipos o soportes	robo de una notebook con datos sin cifrar; conectar un dispositivo en el centro de cómputos
Abuso de privilegios	un usuario *autorizado* usa sus permisos para otros fines (el más difícil de detectar)	consultar el saldo de un famoso; desviar centavos de cada transacción

Consecuencias: destrucción, robo o modificación de activos, violación de la privacidad, interrupción de las operaciones, uso no autorizado de recursos, daño físico a personas (y reputación, multas, juicios).

Controles (mecanismos que reducen las pérdidas esperadas): **preventivos** (impiden: contraseñas, permisos mínimos, validación de entradas, *separación de funciones*, cifrado), **detectivos** (detectan: logs de auditoría, totales de control, alarmas, conciliaciones) y **correctivos** (restauran: backups, plan de continuidad, reversión de transacciones); por alcance, *generales* (todo el entorno: organización, desarrollo, seguridad, operaciones) o *de aplicación* (entrada, procesamiento, salida).

Riesgo de auditoría: que el auditor emita una **opinión equivocada** (dictaminar que el sistema es confiable cuando tiene errores significativos no detectados): $RA = RI \times RC \times RD$, con riesgo **inherente** (errores por la naturaleza de la actividad), **de control** (que los controles no los prevengan ni detecten) y **de detección** (que el auditor no los encuentre). El auditor solo maneja RD: si RI y RC son altos, hace más pruebas sustantivas. **Pasos**: (1) **planificación** (objetivos, alcance, entender el negocio, evaluar riesgos); (2) **relevamiento y evaluación del control interno** (¿hay controles bien diseñados?); (3) **pruebas de controles** (si se confía en ellos: ¿funcionan?); (4) **pruebas sustantivas** (de transacciones y de saldos); (5) **finalización** (opinión, informe con hallazgos y recomendaciones, seguimiento).

En resumen Externo vs. interno; cinco visiones de calidad; modelo fijo (McCall: 11 factores en operación, revisión y transición) o propio (Gilb). Error → falta → falla; bug vs. defecto; densidad y spoilage. ISO: calidad = necesidades enunciadas o implícitas; 9001 con tres colecciones; 9126 con seis características. CMM en cinco niveles sin saltear; CMMI con madurez y capacidad. Auditoría: evidencia sobre activos, integridad, eficacia y eficiencia; abuso, controles, $RA = RI \cdot RC \cdot RD$ y cinco pasos.

12 Hoja de fórmulas

Tema	Fórmula
Scrum	ratio = valor/esfuerzo; suma de puntos del sprint $\leq$ velocity
Soluciones	razón de impactos = $\sum$ impactos de calidad / $\sum$ impactos de recursos
Riesgos	ER = probabilidad $\times$ costo; matriz: prob. $\times$ magnitud (1–6 bajo, 8–9 medio, 12–16 alto)
Estimación	duración = esfuerzo/dedicación; 3 puntos $(O + 4M + P)/6$
CPM	CTe = máxFTe+1; FTe = CTe+d − 1; FTa = mínCTa−1; CTa = FTa−d + 1; V = FTa−CTe+1; MT = $V - d$; ML = mínCTe$_{suc}$−FTe−1
Puntos de función	PFNA = $\sum$ cantidad $\times$ peso; FCT = 0,65 + 0,01 TI; PFA = PFNA $\times$ FCT
COCOMO	$E = aS^bF$ (PM); $D = cE^d$ (meses); $P = E/D$
Puntos objeto	NPO = PO(100 − %reuso)/100; PM = NPO/PROD
Valor ganado	$SV = EV - PV$; $CV = EV - AC$; $SPI = EV/PV$; $CPI = EV/AC$
Calidad	densidad = defectos/KLOC; spoilage = t. corrección/t. desarrollo; Fog = 0,4(pal/or + %complejas)
Equipo (grafo)	densidad = $e/[n(n-1)/2]$; nivel promedio = $2e/n$
Auditoría	RA = RI $\times$ RC $\times$ RD

13 Siglas y preguntas de repaso

AE/AI	archivo externo/interno	LP / GP / GL	líder de proyecto / gerente de proyecto / de línea
AON/AOA	actividad en nodo/flecha	ML / MT	margen libre / total
APC	área de proceso clave (CMM)	PBI	ítem del product backlog
COS	condiciones de satisfacción	PDS	enunciado de definición del proyecto
CTe/FTe	comienzo/fin más temprano	PDM	método de diagramación por precedencia
CTa/FTa	comienzo/fin más tardío	PFNA/PFA	puntos de función no ajustados/ajustados
EE/SE/CE	entrada/salida/consulta externa	POS	enunciado general del proyecto
EV/PV/AC	valor ganado/planificado/costo real	RSGR	reducción, supervisión, gestión
FCE	factores críticos de éxito	TI / FCT	total de influencias / factor de complejidad técnica
JPPS	sesión conjunta de planificación	WBS / EDT	estructura de descomposición del trabajo

1. **Meta vs. objetivo.** La meta es cualitativa (para qué); el objetivo es medible y con plazo.
2. **¿Qué proceso es obligatorio incluso en un proyecto tipo D?** El POS (y aprobación, propuesta, cronograma, reporte de estado, aprobación de entregables, auditoría post-implementación).
3. **¿Por qué no existe un riesgo 100 % probable?** Porque sin incertidumbre es una restricción.
4. **¿Por qué el facilitador de la JPPS no es el LP?** Porque tiene intereses en el resultado y tiende a imponer su visión.
5. **30 días-persona con 3 personas al 50 %.** 30/1,5 = 20 días (si no se pasa el crash point).
6. **CTe** = 5**, FTa** = 14**,** d = 6**.** V = 10, MT = 4.
7. **¿Acortar una tarea con MT** = 5 **acorta el proyecto?** No: no es crítica.
8. **"Listado de alumnos con su promedio": ¿SE o CE?** SE: tiene datos derivados.
9. **Todos los factores en 3: ¿FCT?** TI = 42, FCT = 1,07 (el neutro es TI = 35).
10. **¿**F **en COCOMO básico?** 1. En el intermedio, el producto de 15 conductores.
11. **¿Quién decide si entra un cambio de alcance?** El cliente, entre las alternativas del LP.
12. **¿Qué es un recurso crítico?** El único que cubre un perfil necesario.
13. **SPI** = 1,1 **y CPI** = 0,8**.** Adelantado pero con sobrecosto.
14. **Nueva ley impositiva: ¿qué mantenimiento?** Adaptativo.
15. **ISO 9001 vs. 9126.** 9001: sistema de calidad de la organización (certifica); 9126: producto.
16. **Nivel 2 vs. 3 de CMM.** En el 2 la disciplina es por proyecto; en el 3 hay un proceso estándar de la organización que cada proyecto adapta.
17. **¿Qué componente del riesgo de auditoría controla el auditor?** El de detección.